

Université de Montréal

Application de la programmation en nombres entiers aux
cryptosystèmes post-quantiques basés sur les réseaux

Alexis Lessnick

Département d'informatique et de recherche opérationnelle
Faculté des arts et des sciences

Travail supervisé par Prof. Kim Yu
dans le cadre du cours IFT3150

5 octobre 2026

Note préliminaire et contexte du document

Ce document peut être perçu comme un rapport de recherche étoffé correspondant à l'ensemble de la démarche qui a été entreprise dans le cadre du projet de cryptographie post-quantique que j'ai, Alexis Lessnick, réalisé à l'été 2026. Dans 99% des cas, sa lecture est purement facultative. L'ensemble de la démarche mathématique et des résultats principaux dérivés sont disponible dans sa version analogue plus condensée, aussi disponible sur mon GitHub.

Cependant, cet ouvrage reste pertinent pour quelqu'un désirant s'introduire aux notions mathématiques sous-jacentes à la recherche réalisée dans le cadre de ce projet. Les premières trente pages de ce document constituent ce que je qualifie de «revue de littérature». Elles n'assument aucune compétence préalable en cryptographie, et introduisent le lecteur vers la raison d'exister du domaine. Une progression linéaire depuis les méthodes de cryptographies plus «traditionnelles» vers celles de la cryptographie post-quantique est alors présentée. Un point d'intérêt est forcément accordé aux cryptosystèmes basés sur les réseaux euclidiens, ces derniers constituant le point d'ancrage du travail. Pour terminer, un bref survol des principales notions d'optimisation employées a été rédigé : cela permet alors au lecteur de pleinement passer à la seconde partie du travail avec un esprit plus outillé. Notons alors que cette seconde partie est quasiment identique à celle du rapport condensé. Les deux ouvrages se distinguent donc principalement par la revue de littérature présente dans la version plus garnie.

Table des matières

1	Introduction	7
2	Contexte général	7
2.1	Mise en place du problème et définitions	7
2.2	Cryptographie moderne et systèmes d'actualité	10
2.2.1	Échange de clé de Diffie-Hellman	10
2.2.2	Chiffrement RSA	12
2.3	Faiblesses des systèmes et cassures quantiques	13
3	Réseaux et applications cryptographiques connexes	14
3.1	Définitions	14
3.2	SVP et CVP	16
3.3	Idées de base du chiffrement par les réseaux	18
3.4	L'algorithme LLL	19
3.4.1	Introduction et contexte	19
3.4.2	Description	19
3.4.3	Applications	22
4	Le problème LWE	23
4.1	Définition	23
4.2	LWE et réseaux	24
4.3	SIS : un problème analogue	25
4.4	Sécurité	27
4.5	Implémentation	29
5	Prérequis d'optimisation	30
5.1	Fondements	30
5.2	Algorithmes principaux	32
5.2.1	Branch-and-Bound	32
5.2.2	Méthode des plans sécants	34
6	Réduction principale : $\text{Search-LWE} \leq \text{PQNE}$	36
6.1	Idée générale	36
6.2	Partitionnement des matrices et des vecteurs	37
6.3	Applications linéaires	38
6.4	Construction du problème	39
6.5	Modèle complet	41

7	Implémentation sur Gurobi	42
7.1	Initialisation	42
7.2	Créations d’instances, validation et script final.	44
7.3	Tests et résultats préliminaires	45
7.4	Performance face aux cryptosystèmes actuels	48
8	Améliorations	52
8.1	Alignement aux standards industriels	52
8.2	Réduction directe de la matrice W	55
8.3	Contruction algébrique alternative	56
8.3.1	Modélisation mathématique	56
8.3.2	Étude de performance	60
9	Conclusion	63
9.1	Travail accompli	63
9.2	Avenues futures	64
	Annexes	69
A	Implémentation de la réduction LWE	69

PARTIE 1

Revue de littérature

1 Introduction

L'information a toujours été un pilier de la société humaine et nombreux sont les travaux ayant été entrepris dans son domaine. Pendant longtemps, des analyses plus classiques telles l'étude de textes littéraires et philosophiques prédominaient. Certains tentaient de comprendre autrui via le fond caché d'oeuvres parfois poétiques. Les subtiles nuances d'un adjectif, les métaphores et les rimes se combinaient pour donner un sens tout autre à la façade se dégageant d'une première lecture. D'une certaine façon, la cryptographie y est analogue, l'idée générale y étant très similaire. En se concentrant sur son côté mathématique, le domaine est beaucoup plus cartésien, et il est rare qu'il n'existe plus d'une interprétation. Cependant, son but y reste le même : on souhaite que seuls ceux dont on le désire puissent réellement comprendre le message que l'on essaie de dissimuler.

Le présent travail débutera en premier lieu par une introduction sommaire à la cryptographie mathématique et aux systèmes qui sont actuellement les standards de l'industrie. Les failles de ceux-ci compte tenu de l'existence d'un ordinateur quantique fonctionnel y seront présentées et des alternatives seront proposées. Une attention particulière sera accordée aux réseaux (*lattices*) qui constitueront la base théorique de la deuxième section du compte-rendu. Cette seconde moitié traitera des manières d'utiliser la programmation en nombres entiers afin de briser les systèmes cryptographiques basés sur les réseaux. L'un des principaux problèmes que l'on croit être résistant aux technologies quantiques, soit *Learning With Errors* (ou *LWE*), sera étudié et attaqué via Gurobi, un solveur de problèmes d'optimisation.

2 Contexte général

2.1 Mise en place du problème et définitions

Pour commencer, il est impératif de se doter de vocabulaire et de définitions de base [30]. On pose le problème de cryptographie comme suit : supposons que l'on a deux individus, que l'on nommera Alice et Bob. Ces derniers désirent communiquer via un canal sur lequel Ève, une personne tierce, est à l'écoute. Petit problème : Ève est malveillante et ne doit pas pouvoir comprendre les communications partagées. On désire donc voiler l'information de manière à ce qu'Ève ne puisse en comprendre le sens. On appelle «texte clair» le texte que l'on désire cacher de l'attaquante et l'on nomme «texte chiffré» le texte accessible à cette dernière. On appelle la «clé» l'information critique qui permet de passer du texte clair au texte chiffré, et vice-versa. Pendant très longtemps, les systèmes cryptographiques supposaient qu'Alice et Bob s'étaient déjà rencontrés pour se mettre d'un commun accord sur une clé qui devait absolument rester privée. Si Ève obtenait cette dernière, alors la sécurité du système cryptographique était réduite à néant. Il n'était donc pas possible pour Alice et Bob de choisir la clé en communiquant sur le canal que l'on a posé au début, car Ève en soutirerait sa valeur.

En 1976, Diffie et Hellman posent dans une publication révolutionnaire les bases d'un système de cryptographie à clé publique [5]. Cette façon de procéder est intéressante puisqu'elle suppose qu'Alice et Bob peuvent réussir à communiquer la clé de chiffrement via le canal partagé avec Ève sans que cette dernière ne puisse en déduire facilement la valeur. Dans un tel système, il faut travailler avec une *fonction à sens unique*, c'est-à-dire une fonction facile à calculer avec un inverse difficile à calculer. Dans les systèmes à clé publique, ces fonctions ont une porte de secours, une bribe d'information auxiliaire qui permet de calculer l'inverse facilement. Il est intéressant de noter que l'on suppose que les fonctions à sens unique existent, mais que leur existence n'a toujours pas été formellement prouvée. La sécurité de ces systèmes repose donc sur l'hypothèse que l'inverse de ces fonctions est difficile à calculer¹.

Pour conclure cette mise en place, posons quelques concepts pertinents :

Définition 1. Soit a et b des entiers où $b \neq 0$. On dit que b *divise* a s'il existe un entier c tel que $a = bc$. On écrira alors $b \mid a$ si c'est le cas, et $b \nmid a$ si ce n'est pas le cas.

Définition 2 (Plus grand commun diviseur). Le *plus grand commun diviseur* de deux entiers a et b est le plus grand entier positif d tel que $d \mid a$ et $d \mid b$. On notera $d = \text{pgcd}(a, b)$.

Définition 3 (Algorithme d'Euclide). L'*Algorithme d'Euclide* est un algorithme exécutable en temps polynomial qui permet de trouver le *pgcd* de deux entiers a et b .

Définition 4 (Nombres relativement premiers). Deux entiers a et b sont *relativement premiers* si $\text{pgcd}(a, b) = 1$.

Proposition 1. Soit a un entier. Alors $a \cdot b \equiv 1 \pmod{m}$ pour un entier b si et seulement si a et m sont relativement premiers. On appelle b l'inverse de a modulo m .

Définition 5 (Algorithme d'Euclide étendu). L'*Algorithme d'Euclide étendu* est un algorithme exécutable en temps polynomial qui, pour deux entiers relativement premiers a et m , retourne l'inverse de $a \pmod{m}$, dont on a posé la définition dans la proposition 1 plus haut.

Définition 6 (Algorithme d'exponentiation rapide). L'*Algorithme d'exponentiation rapide* est un algorithme exécutable en temps polynomial qui permet de calculer de très grandes puissances d'un nombre. Il est important de noter que l'algorithme s'applique sous n'importe quel système multiplicatif, et donc qu'il peut calculer rapidement $a^x \bmod p$, où $a, x, p \in \mathbb{N}^*$.

Définition 7. On note $\mathbb{Z}/m\mathbb{Z} = \{0, 1, 2, \dots, m-1\}$ et on appelle cet ensemble l'anneau des entiers modulo m . Bien que cela soit la notation officielle propre à la théorie des groupes, on utilise plus souvent $\mathbb{Z}_q$ par souci de clarté en cryptographie, où $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}$. C'est la convention qui sera utilisée dans ce travail.

1. Si l'on arrive à prouver que les fonctions à sens unique existent, alors $P \neq NP$

Il ne nous reste plus que deux théorèmes à proposer avant d'exposer les méthodes de cryptographie modernes. Nous ne les prouverons pas, mais les preuves de ces derniers sont disponibles dans [30].

Théorème 1 (Petit théorème de Fermat). Soit p un nombre premier et soit a un entier quelconque. Alors :

$$a^{p-1} \equiv \begin{cases} 1 & (\text{mod } p) \quad \text{si } p \nmid a, \\ 0 & (\text{mod } p) \quad \text{si } p \mid a. \end{cases}$$

Remarque 1. Le petit théorème de Fermat nous permet de rapidement tester la primalité d'un très grand nombre. Prenons par exemple le nombre 15485207. On pose le problème sous forme du petit théorème de Fermat avec $a = 2$ et $p = 15485207$. On a donc :

$$2^{p-1} = 2^{15485206} \equiv 4136685 \pmod{15485207}$$

On remarque que la congruence a été calculée rapidement par l'algorithme d'exponentiation énoncé plus haut, et qu'en observant le problème, le petit théorème de Fermat n'est pas satisfait. On en déduit donc que 15485207 n'est pas premier. Cependant, des faux-positifs existent : on ne peut donc pas s'assurer que le nombre testé est premier, car certains nombres composés satisferont aux conditions du théorème. En pratique, on utilise donc l'algorithme probabiliste plus complexe de Miller-Rabin [23] se basant sur le petit théorème de Fermat.

Glissons une petite définition avant d'introduire le second théorème :

Définition 8. L'ordre d'une congruence $a^x \pmod{m}$ est le plus petit exposant $x \geq 1$ tel que $a^x \equiv 1 \pmod{m}$.

Théorème 2 (Théorème des racines primitives). Soit q un nombre premier. Alors il existe un élément $g \in \mathbb{Z}_q^*$ tel que ses puissances donnent tous les éléments de $\mathbb{Z}_q^*$. En d'autres mots :

$$\mathbb{Z}_q^* = \{1, g, g^2, \dots, g^{q-2}\}$$

On appelle les éléments avec cette propriété les générateurs de $\mathbb{Z}_q^*$. Ils se caractérisent par un ordre de $q - 1$ sous le système $g^x \equiv 1 \pmod{q}$.

Exemple 1. Dans $\mathbb{Z}_5^*$, le nombre 2 est un élément générateur. En calculant ses puissances successives modulo 5, on obtient :

$$2^0 \equiv 1, \quad 2^1 \equiv 2, \quad 2^2 \equiv 4, \quad 2^3 \equiv 3 \pmod{5}.$$

Les 4 éléments de $\mathbb{Z}_5^*$ sont visiblement tous générés.

Remarque 2. Quand q est un grand nombre, alors $\mathbb{Z}_q^*$ a beaucoup d'éléments générateurs. Il est possible de vérifier que les éléments générateurs de $\mathbb{Z}_{17}^*$ sont $\{3, 5, 6, 7, 10, 11, 12, 14\}$ par exemple.

2.2 Cryptographie moderne et systèmes d'actualité

2.2.1 Échange de clé de Diffie-Hellman

Compte tenu des prérequis mathématiques explicités à la section du haut, il devient maintenant possible d'introduire l'échange de clé de Diffie et Hellman (que l'on abrégera par DH), soit le premier système de cryptographie à clé publique jamais proposé. Bien que ce protocole ne soit pas utilisé en industrie à cause de la taille trop imposante des clés impliquées, des variantes de ce dernier sur les courbes elliptiques le sont. Il reste donc pertinent de présenter le problème classique car les vulnérabilités de cette classe de protocoles face à l'ordinateur quantique sont identiques, et parce que cela permet de mieux retracer l'évolution de la cryptographie moderne.

DH repose sur le problème de résolution du logarithme discret, que l'on définit comme suit :

Définition 9. Soit g un élément générateur de $\mathbb{Z}_q^*$ avec q un nombre premier et soit h un élément quelconque de $\mathbb{Z}_q^*$. Alors le problème du logarithme discret consiste à trouver un exposant x tel que

$$g^x \equiv h \pmod{p}$$

L'entier x est alors appelé le logarithme discret de h en base g .

Comme g est un élément générateur, alors on peut garantir qu'il existe forcément un exposant x qui résout le problème du logarithme discret. En analysant le problème davantage, on peut remarquer que si x est une solution, alors $\forall k \in \mathbb{N}, x + k(p-1)$ en est aussi une, et donc qu'il existe une infinité de solutions. À titre de préambule, avançons que pour un ordinateur classique, cela n'affecte en rien la difficulté du problème. Cependant, c'est directement en lien avec la vulnérabilité qu'exploite l'ordinateur quantique. Le sujet sera élaboré dans la prochaine section.

Il n'existe pour l'instant pas d'algorithme classique qui résout le problème de manière efficace. Un algorithme naïf essayant de résoudre le problème par force brute testant toutes les valeurs possibles de x de 0 jusqu'à q dépendrait directement de la taille de l'encodage $q \approx 2^k$ de notre nombre premier. L'algorithme s'exécuterait donc en temps $\mathcal{O}(2^k)$, ce qui n'est pas du tout efficace.

Tout compte fait, énumérons les étapes entreprises lors de l'échange de clé de Diffie et Hellman.

Pour initialiser le protocole, Alice et Bob se mettent d'accord sur un nombre premier p , en utilisant un algorithme tel que Miller-Rabin. On cherche ensuite un générateur g de $\mathbb{Z}_p^*$. Ces étapes sont réalisables de manière efficace par un ordinateur [16]. On note que ces valeurs sont

publiques, et donc qu'elles sont accessibles à Ève. Ensuite, Alice se choisit un entier secret a , et Bob se choisit un entier secret b . Alice calcule

$$A \equiv g^a \pmod{p}$$

et Bob calcule de manière analogue

$$B \equiv g^b \pmod{p}$$

Ils s'échangent ensuite ces valeurs. On remarque qu'Ève a accès à celles-ci. Pour conclure, Alice calcule

$$A' \equiv B^a \pmod{p}$$

tandis que Bob calcule

$$B' \equiv A^b \pmod{p}$$

On remarque que ces valeurs sont identiques, car

$$A' \equiv B^a \equiv (g^b)^a \equiv (g^a)^b \equiv A^b \equiv B' \pmod{p}$$

La clé est donc cette valeur commune, qu'on appellera ici k .

Bien qu'Ève ait accès à toutes les valeurs échangées, elle se doit d'obtenir l'entier secret de soit Alice soit Bob pour trouver k . On remarque donc que le problème de Diffie-Hellman n'est pas plus difficile que le problème du *DLP*. En effet, avec l'une des valeurs interceptées, choisissons ici A , Ève peut tenter de résoudre l'instance $A \equiv g^a \pmod{p}$ du problème du DLP. Admettons qu'elle le résolve efficacement, alors elle n'aurait qu'à trouver la clé $k \equiv B^a \pmod{p}$, que l'on sait déjà qu'elle peut calculer en temps polynomial via l'algorithme d'exponentiation rapide.

Exemple 2. Donnons une simulation de *DH* tirée directement de [30]. Forcément, par souci de clarté, les valeurs p et g choisies seront plus petites que les valeurs devant réellement être utilisées.

Alice et Bob se mettent d'accord sur $p = 941$ et sur $g = 627$. Alice choisit $a = 347$ et calcule ensuite $A = 390 \equiv 627^{347} \pmod{941}$. De manière analogue, Bob choisit $b = 781$ et calcule $B = 691 \equiv 627^{781} \pmod{941}$. Ils s'échangent ensuite ces valeurs, et donc A, B, p, g sont maintenant des valeurs publiques. Seuls a et b restent secrets. Les deux calculent ensuite de leurs côtés

$$470 \equiv 627^{347 \cdot 781} \equiv A^b \equiv B^a \pmod{941}$$

Ils obtiennent donc la clé secrète $k = 470$, tandis que Ève est contrainte à résoudre soit

$$627^a \equiv 390 \pmod{941} \text{ ou } 627^b \equiv 691 \pmod{941}$$

Pour Ève, il n'existe malheureusement pas d'algorithme qui puisse calculer ces congruences efficacement.

2.2.2 Chiffrement RSA

Il est naturel d'enchaîner avec l'un des plus anciens systèmes de cryptographie à clé publique toujours pertinent et utilisé aujourd'hui, soit le système de cryptographie *RSA*, pour Rivest, Shamir et Adleman. Son gage de sécurité découle de la difficulté de résoudre efficacement des équations de la forme

$$x^e \equiv c \pmod{N}$$

ce qui est quasiment identique au problème du logarithme discret, à l'exception que cette fois-ci, les valeurs de e, c et N sont connues, et la valeur x de la base est celle recherchée.

Avant de présenter ledit système, avançons un résultat dont nous omettrons la preuve. Elle se trouve dans [30] au besoin.

Proposition 2. Soit p et q deux nombres premiers distincts et e un entier plus grand que 1 tels que

$$\text{pgcd}(e, (p-1)(q-1)) = 1.$$

Comme ces valeurs sont relativement premières, alors par la proposition 1 que l'on a avancée plus haut, e admet un inverse modulo $(p-1)(q-1)$ que l'on pose comme

$$de \equiv 1 \pmod{(p-1)(q-1)}.$$

Ces valeurs étant posées, il est possible de prouver que la congruence $x^e \equiv c \pmod{pq}$ a une solution unique, soit $x \equiv c^d \pmod{pq}$. C'est l'astuce principale de cette méthode cryptographique.

Le système *RSA* va donc comme suit : Bob choisit deux nombres premiers secrets p et q . Il calcule son module public $N = p \cdot q$. Via l'algorithme d'Euclide, Bob trouve un exposant e tel que $\text{pgcd}(e, (p-1)(q-1)) = 1$. Alice veut envoyer un message chiffré à Bob. Supposons que ce message peut être converti en un nombre entier, que l'on nomme m , respectant $1 \leq m < N$. Alice calcule donc $c \equiv m^e \pmod{N}$, et envoie ce c à Bob comme message chiffré.

Comme Bob connaît p et q , alors il peut aisément calculer l'unique inverse d de e modulo $(p-1)(q-1)$ par l'algorithme d'Euclide étendu. Avec cet exposant d , il peut transformer la congruence $m^e \equiv c \pmod{N}$ en utilisant la proposition 2, comme il sait que sa solution unique est $m \equiv c^d \pmod{N}$. L'expression $c^d \pmod{N}$ est calculable efficacement via l'algorithme d'exponentiation rapide, et donc Bob peut trouver la valeur de m rapidement.

Ève, quant à elle, est prise au dépourvu. Elle a à sa disposition c, e et N . Elle pourrait facilement

calculer l'inverse de e modulo N , mais cela ne serait pas pertinent car la clé d est l'inverse de e modulo $(p-1)(q-1)$. Elle doit donc absolument trouver les valeurs de p et de q pour trouver d , et pour ce faire, elle doit factoriser N . Par le théorème fondamental de l'arithmétique, cette factorisation est unique. En arrivant à factoriser N , elle tomberait forcément sur p et q , et il lui serait ensuite facile de déchiffrer le message comme l'a fait Bob. Cependant, il n'existe pas d'algorithme efficace pour calculer cette factorisation, et donc elle ne peut rapidement interpréter le message envoyé.

2.3 Faiblesses des systèmes et cassures quantiques

Compte tenu de la nature abstraite et de la très grande quantité d'information requise à la compréhension même sommaire de la matière à l'étude, nous ne ferons qu'un survol des idées.

À travers la dernière section, nous avons présenté deux cryptosystèmes différents. Remarquons que l'échange de clé de Diffie-Hellman était un système incomplet : il ne s'occupe pas du chiffrement ni du déchiffrement, mais seulement de la génération d'une clé de chiffrement commune à Alice et à Bob. Le second système, *RSA*, était plus complet. Cependant, pour pouvoir communiquer sécuritairement, il était impératif qu'Alice et Bob génèrent chacun leur propre clé. Malgré ces petites différences, les faiblesses de ces protocoles face à l'ordinateur quantique sont similaires.

En 1994, Peter Shor propose deux algorithmes quantiques différents [29]. Le premier permet de trouver en temps polynomial une factorisation en nombres premiers, et le second résout aussi efficacement le problème du logarithme discret. Quelques mois plus tard, Alexei Kitaev propose une mise en commun de ces deux algorithmes, comme leur fonctionnement sous-jacent est identique [11]. Tant le *DLP* que le problème de factorisation de *RSA* sont réduisibles au problème du sous-groupe caché, communément abrégé en *HSP* [7].

Très informellement, il est possible de voir le *HSP* comme un problème d'identification de période d'une fonction définie sur un groupe G , la période en elle-même constituant le sous-groupe recherché H . Pour trouver ladite période, il faut évaluer de façon répétée une fonction retournant un résultat identique si et seulement si la valeur qu'on lui passe en entrée est décalée par un élément du sous-groupe. Donnons un exemple très simple du problème :

Exemple 3. Considérons le groupe $G = \mathbb{Z}$ et le sous-groupe caché $H = 10\mathbb{Z}$ (les multiples de 10). On définit une fonction oracle $f(x) = x \bmod 10$, qui extrait le chiffre des unités.

Pour une entrée $x = 3$, on a $f(3) = 3$. Si l'on applique un décalage par un élément du sous-groupe, soit $20 \in H$, la fonction donne un résultat identique : $f(3 + 20) = f(23) = 3$. À l'inverse, un décalage par un élément hors du sous-groupe, soit $4 \notin H$, génère un résultat distinct : $f(3 + 4) = f(7) = 7 \neq 3$.

L'ordinateur quantique a le pouvoir de calculer cette fonction f en superposition : en une

seule itération de f , il obtient les évaluations $f(x)$ pour chacun des éléments x du groupe G . Cependant, c'est lors de la mesure que tout s'effondre toujours vers un seul groupe de valeurs. À titre de comparaison, on pourrait mesurer, dans le groupe que l'on a posé dans l'exemple plus haut, l'ensemble des éléments dont le chiffre des unités est trois. On aurait donc $H_1 = \{\dots -23, -13, -3, 3, 13, 23\dots\}$. Sur cet ensemble, la prochaine étape est d'y appliquer la transformée de Fourier quantique (QFT). Des informations sur la période y seront extraites, mais il ne sera pas possible de la découvrir directement. Ce bloc de calculs en superposition, de mesure et de QFT sera répété un nombre polynomial de fois, jusqu'à ce que l'ordinateur quantique ait retiré assez d'informations sur la structure du sous-groupe. Il enverra ces informations à un ordinateur classique qui pourra finalement trouver la période.

Le paragraphe précédent est la fondation même des algorithmes avec lesquels est apparue la menace quantique. Ils fonctionnent à très basse échelle pour l'instant : les nombres ayant déjà été factorisés par des instructions de ce type n'ont pour le moment jamais dépassé 35 [32]. Cependant, cette limite demeure pour l'instant matérielle, et non logicielle. Remarquons pour conclure que les instances de HSP sur lesquelles l'ordinateur quantique offre théoriquement un avantage tombent quasiment toutes dans la même catégorie : ce sont des problèmes où le groupe G à l'intérieur duquel on recherche H est abélien. En effet, la QFT nécessite une certaine commutativité pour retourner des résultats exploitables. L'étude des groupes non-abéliens s'avère donc fort intéressante, permettant la découverte potentielle de méthodes de chiffrement contre l'ordinateur quantique.

3 Réseaux et applications cryptographiques connexes

3.1 Définitions

Avec l'avènement potentiel de protocoles de déchiffrement quantiques efficaces, il fut impératif pour la communauté scientifique de consacrer de l'énergie au développement et à l'analyse de systèmes cryptographiques pouvant résister à ces avancées. Il est important de faire la distinction entre la cryptographie quantique et post-quantique : la cryptographie quantique est basée sur des notions physiques (on peut penser au protocole d'échange de clé *BB84*), tandis que la cryptographie post-quantique est purement mathématique et plus facile d'approche. Nous quitterons donc le monde quantique dans les paragraphes à suivre, mais gardons en tête la motivation derrière les techniques de chiffrement qui seront proposées. À ce jour, nombreux sont les systèmes cryptographiques que l'on croit résister tant à l'ordinateur classique qu'à l'ordinateur quantique. On peut penser à la cryptographie basée sur les hachages, sur les codes, sur les réseaux euclidiens ou même sur les équations quadratiques multivariées. Nous nous concentrerons sur les réseaux euclidiens lors de notre analyse, dont on présente la définition comme suit :

Définition 10. (Réseau euclidien) Soit $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{R}^m$ un ensemble de vecteurs linéairement

indépendants. Le réseau euclidien, que l'on appellera simplement réseau, est l'ensemble des combinaisons linéaires de $\mathbf{v}_1, \dots, \mathbf{v}_n$ à coefficients en $\mathbb{Z}$. On a donc

$$L = \{a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \dots + a_n\mathbf{v}_n : a_1, a_2, \dots, a_n \in \mathbb{Z}\}$$

avec L la lettre qui désigne notre réseau, comme ce même réseau est communément appelé *lattice* en anglais.

Remarque 3. Pour des raisons de stabilité numérique, nous travaillerons exclusivement avec des réseaux dont les vecteurs de base ont des composantes entières, où $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{Z}^m$. Cela nous permettra de ne générer que des réseaux entiers, soit des réseaux dont tous les vecteurs/les points auront forcément des composantes entières (car ce sont des combinaisons linéaires à coefficients entiers de vecteurs de base composés uniquement d'entiers).

Exemple 4. Pour illustrer la structure d'un réseau, considérons un exemple dans $\mathbb{R}^2$. On remarque que les combinaisons linéaires du réseau sont restreintes aux entiers $\mathbb{Z}$ par les conditions que l'on a posées plus tôt. C'est donc un espace discret de points isolés plutôt qu'un espace continu.

Définissons une base $B = \{b_1, b_2\}$ constituée des deux vecteurs linéairement indépendants suivants :

$$b_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

Le réseau engendré par cette base, noté $\mathcal{L}(B)$, peut être visualisé par la grille suivante :

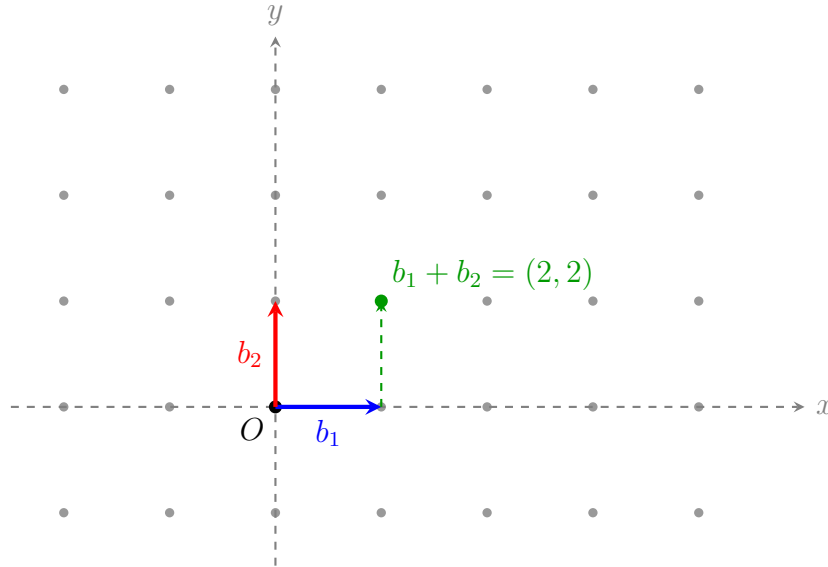


FIGURE 1 – Représentation géométrique du réseau $\mathcal{L}(B)$ engendré par b_1 et b_2 ².

2. Notons qu'il s'étendrait à l'infini si des contraintes d'espace ne nous étaient pas imposées.

Avec un œil affûté, on peut remarquer que la base proposée n'est pas la seule base qui génère ce réseau. Par exemple, la base $B' = \{b_3, b_4\}$ constituée des vecteurs suivants génère exactement la même grille de points :

$$b_3 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad b_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

La base B initiale que nous avons proposée était orthogonale : cela nous permettait d'aisément générer la visualisation du réseau qu'elle engendrait. À titre d'intuition pour les prochaines sections, on remarque que bien que la base B' génère exactement la même grille carrée que B , déterminer les coefficients nécessaires à la création de cette dernière semble être beaucoup plus complexe.

Proposition 3. Sans le prouver, avançons que deux bases d'un réseau L sont reliées par une matrice à coefficients entiers et à déterminant égal à ± 1 . Nous appellerons cette matrice une *matrice de passage*.

Il ne nous reste que quelques idées à poser avant de rentrer dans le vif du sujet.

Définition 11. (Domaine fondamental) Soit L un réseau et $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ une base de L . Alors le *domaine fondamental* de L est défini comme

$$\mathcal{F} = \mathcal{F}(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n) = \{t_1\mathbf{v}_1 + t_2\mathbf{v}_2 + \dots + t_n\mathbf{v}_n : 0 \leq t_i < 1\}$$

que l'on peut interpréter comme l'espace couvert par les vecteurs de la base de L .

Définition 12. Le *volume* du domaine fondamental est défini comme

$$\text{Vol}(\mathcal{F}) = \det(L) = |\det(\mathcal{F}(\mathbf{v}_1, \dots, \mathbf{v}_n))|$$

qui nous sera une quantité importante lors de l'analyse des problèmes de réseaux à venir.

Remarque 4. Cette quantité $\text{Vol}(\mathcal{F}) = \det(L)$ est indépendante de la base choisie parmi les bases engendrant un même réseau. La preuve de cette affirmation se trouve dans [30].

3.2 SVP et CVP

Les deux problèmes fondamentaux liés aux réseaux sont les suivants :

Définition 13. (*SVP*) Le problème du vecteur le plus court, abrégé *SVP* pour *Shortest Vector Problem*, est le problème de recherche du vecteur à la norme non-nulle la plus petite dans un réseau L .

Définition 14. (*CVP*) Le problème du vecteur le plus proche, abrégé *CVP* pour *Closest Vector Problem*, est le problème de recherche du vecteur $\mathbf{v} \in L$ le plus proche d'un autre vecteur $\mathbf{w} \in R^m$ donné qui n'est pas dans L . Autrement dit, on cherche le vecteur v qui minimise $\|\mathbf{w} - \mathbf{v}\|$.

CVP est NP -Difficile et SVP l'est aussi, mais seulement sous certaines conditions. Ces problèmes ont de nombreuses variantes tant en pratique qu'en théorie : ils constituent une base pour les problèmes de réseaux. Le théorème qui va suivre permettra de garantir une solution à ceux-ci.

Définition 15. Soit $L \subseteq \mathbb{Z}^n$ un réseau entier. Pour tout $i \in [1, n]$, le i^{eme} minimum successif $\lambda_i(L)$ est défini comme le plus petit nombre réel r tel que L contient i vecteurs linéairement indépendants de taille maximale r [15]. La figure ci-dessous donne une interprétation visuelle de ces minimums.

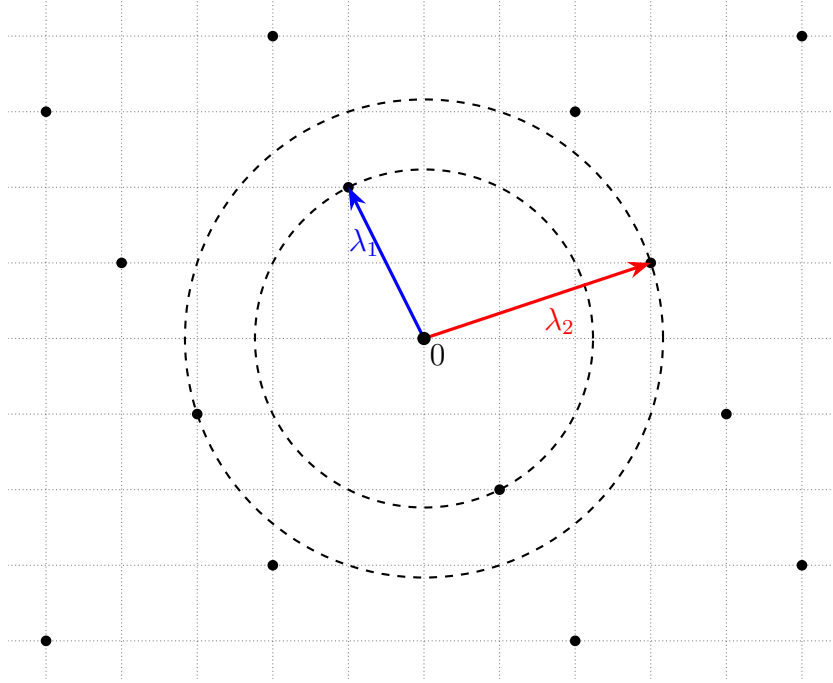


FIGURE 2 – Deux premiers minimums successifs λ_1 et λ_2 d'un réseau.

Charles Hermite tenta de borner supérieurement le premier minimum successif, comme il correspond directement à la taille du plus petit vecteur du réseau. Hermann Minkowski vint ajouter aux travaux d'Hermite et proposa le résultat suivant, à la borne supérieure beaucoup plus resserrée en haute dimension.

Théorème 3. (Théorème de Minkowski) Soit $L \subseteq \mathbb{Z}^n$ un réseau plein-rang³. La taille du plus petit vecteur du réseau, $\lambda_1(L)$, satisfait nécessairement la relation suivante :

$$\lambda_1(L) \leq \sqrt{n} \det(L)^{1/n}.$$

Ce théorème est puissant, car il nous garantit que le plus petit vecteur du réseau existe et se situe dans un espace à distance maximale de $\sqrt{n} \det(L)^{1/n}$ de l'origine, raffinant ainsi particulièrement l'espace de recherche.

3. Comme une matrice : le nombre de vecteurs linéairement indépendants de la base est égal à la dimension du réseau.

3.3 Idées de base du chiffrement par les réseaux

Nous proposons ici un système cryptographique très simplifié, mais analogue au cryptosystème GGH (lui aussi déjà désuet), qui agit comme précurseur du domaine.

Pour commencer, avançons qu'une bonne base est une base raisonnablement orthogonale : comme on l'a vu plus tôt avec notre exemple de réseau, il semble plus facile de calculer les vecteurs d'un réseau si les vecteurs de sa base sont orthogonaux. Par la définition qui suit, nous proposons une mesure de quantification de l'orthogonalité d'une base.

Définition 16. (Ratio d'Hadamard) Soit $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ une base d'un réseau quelconque L . Le *ratio d'Hadamard* de la base est défini comme étant la quantité suivante :

$$\mathcal{H}(\mathcal{B}) = \left(\frac{\det(L)}{\|\mathbf{v}_1\| \cdot \|\mathbf{v}_2\| \dots \|\mathbf{v}_n\|} \right)^{1/n}.$$

On a $0 < \mathcal{H}(\mathcal{B}) \leq 1$. Plus le ratio tend vers 1, plus la base est orthogonale, et donc on a une «bonne» base, et plus le ratio tend vers 0, plus les vecteurs sont dans le même axe, et donc plus la base est «mauvaise».

Reprenons nos personnages préférés, soit Alice et Bob, et notre vilaine préférée, Ève. Supposons que tous possèdent un algorithme efficace qui détermine le vecteur d'un réseau $L \in \mathbb{Z}^n$ le plus proche d'un point arbitraire $x \in \mathbb{R}^n$, mais que cet algorithme ne fonctionne que si les vecteurs de la base du réseau sont quasi-orthogonaux. C'est l'algorithme de Babai [3]. Alice commence donc par choisir une base de vecteurs relativement orthogonaux : elle vérifie leur orthogonalité via le ratio d'Hadamard, et s'assure qu'ils ne sont pas trop grands pour le réseau L qu'ils engendrent en les comparant au théorème de Minkowski. Cette base, lorsque combinée en une matrice, sera appelée V . Alice génère ensuite une seconde matrice U de même dimension telle que $\det(U) = \pm 1$. Comme on l'a vu plus tôt, la matrice $W = UV$ constituera une nouvelle base pour L . Alice s'assure que cette base est mauvaise en calculant son ratio d'Hadamard, et offre alors W comme clé publique, tout en gardant V comme clé privée.

Supposons que Bob veuille envoyer un message à Alice. Il calculera alors le vecteur $\mathbf{e} = \mathbf{m}W + \mathbf{r}$, où $\mathbf{e}$ est le vecteur chiffré, $\mathbf{m}$ est le vecteur de message, et $\mathbf{r}$ est une petite perturbation dans le système. Le vecteur $\mathbf{e}$ n'est donc pas un vecteur du réseau L , mais bien plutôt un vecteur dans $\mathbb{Z}^n$ quelconque. On remarque donc que le problème se transforme en une instance de CVP. Alice détenant la bonne base V peut appliquer l'algorithme de Babai et retrouver le point le plus proche de $\mathbf{e}$ en temps polynomial. Elle retrouve donc $\mathbf{m}W$, et en multipliant cette quantité par W^{-1} , elle retrouve $\mathbf{m}$. Ève, quant à elle, reste coincée : sans bonne base, elle ne peut appliquer l'algorithme de Babai, et donc elle ne peut résoudre le CVP. Son vrai problème est donc de trouver une dite bonne base : malheureusement, une excellente option s'offre à elle.

3.4 L'algorithme LLL

3.4.1 Introduction et contexte

En 1982, Lenstra, Lenstra et Lovász publient un article dans lequel ils présentent un algorithme révolutionnaire[12]. Ce dernier finira par prendre leur nom, soit LLL ou L^3 . La chronologie exacte du développement de l'algorithme est peu claire, compte tenu du fait que leur recherche initiale n'avait rien à voir avec la programmation en nombres entiers (et forcément encore moins en cryptographie). Certains disent que l'idée qui les a poussés à chercher sur le sujet provient de Peter Van Emde Boas. Ce dernier est entré dans le bureau d'Hendrik Lenstra avec comme objectif la résolution du problème suivant [31] :

Question 1. Soit un triangle à coordonnées rationnelles dans le plan. Existe-t-il un point à coordonnées entières existant dans ce même triangle ?

Lenstra avança que ce problème pouvait être résolu avec l'algorithme de réduction de réseau de Gauss en dimension deux, et trouva initialement le problème très peu pertinent. Van Emde Boas quitta donc son bureau, et Lenstra se mit à réfléchir. En un bref 45 minutes, il s'aperçut que le problème pouvait être généralisé en dimension n . Il publia alors en 1981 l'article *Integer Programming with a fixed number of variables* [13], où il argumenta qu'en dimension fixée, les programmations en nombres entiers sont résolubles polynomialement. Nous y ferons référence plus tard. Bien que l'algorithme s'exécute théoriquement en temps polynomial, les choses n'étaient pas aussi faciles en pratique. Le degré du polynôme étant exponentiel selon n , la méthode devint rapidement très lourde en haute dimension. C'est à ce moment que László Lovász intervint et ajouta une relaxation à l'algorithme. Le problème fut adapté au problème de factorisation de polynômes d'Arjen Lenstra, le frère d'Hendrik et naquit l'algorithme que l'on connaît aujourd'hui. Posons donc les bases pour ce dernier.

3.4.2 Description

Supposons que l'on possède une mauvaise base $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$. On cherche à la transformer en bonne base pour que le ratio d'Hadamard (voir la Définition 16) tende vers 1. Rappelons le fonctionnement du processus de Gram-Schmidt, si $\mathcal{B}$ est notre base d'entrée et que $\mathcal{B}^* = \{\mathbf{v}_1^*, \dots, \mathbf{v}_n^*\}$ est notre base de sortie, qui sera parfaitement orthogonale. Tout débute par $\mathbf{v}_1^* = \mathbf{v}_1$, et ensuite pour $i \geq 2$, on a

$$\mathbf{v}_i^* = \mathbf{v}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{v}_j^* \quad \text{où} \quad \mu_{i,j} = \frac{\mathbf{v}_i \cdot \mathbf{v}_j^*}{\|\mathbf{v}_j^*\|^2}.$$

L'espace continu engendré par $\mathcal{B}$ et par $\mathcal{B}^*$ est le même par définition du procédé, cependant, $L(\mathcal{B}) \neq L(\mathcal{B}^*)$. $\mathcal{B}^*$ est donc un référent idéal sur lequel est basé *LLL*, car son ratio d'Hadamard est de 1 compte tenu du fait que $\det(L) = \prod_{i=1}^n \|\mathbf{v}_i^*\|$ [30]. Nous nous basons sur cette dernière

pour définir un concept fondamental au succès de l'algorithme *LLL*.

Définition 17. Soit $\mathcal{B}$ une base quelconque d'un réseau L et soit $\mathcal{B}^*$ la base de Gram-Schmidt qui y est associée. Alors $\mathcal{B}$ est dite *LLL-réduite* si et seulement si elle satisfait les deux conditions suivantes :

1. **Condition de taille**

$$|\mu_{i,j}| = \frac{|\mathbf{v}_i \cdot \mathbf{v}_j^*|}{\|\mathbf{v}_j^*\|^2} \leq \frac{1}{2}$$

pour tout $1 \leq j < i \leq n$.

2. **Condition de Lovász**

$$\|\mathbf{v}_i^*\|^2 \geq \left(\frac{3}{4} - \mu_{i,i-1}^2 \right) \|\mathbf{v}_{i-1}^*\|^2$$

pour tout $1 < i \leq n$.

Une telle base est particulièrement désirable puisqu'elle peut résoudre *apprSVP*, soit le problème de recherche d'un vecteur qui n'est ni nul ni plus grand que $\psi(n)$ fois le vecteur le plus court du réseau, à un facteur $\psi(n) = 2^{(n-1)/2}$. De manière analogue, elle pourrait résoudre *apprCVP* à un facteur exponentiel de $(2/\sqrt{3})^n$ [17]. Proposons maintenant l'algorithme *LLL* en sa totalité. Nous utilisons la notation $\lfloor x \rfloor$ pour désigner l'arrondissement à l'entier le plus proche.

Algorithme 1 : Algorithme LLL

Entrée : Une base $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ d'un réseau L

Sortie : Une base LLL-réduite $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

$\mathbf{v}_1^* \leftarrow \mathbf{v}_1;$

$k \leftarrow 2;$

tant que $k \leq n$ **faire**

pour $j = k - 1$ **à** 1 **faire**

$\mathbf{v}_k \leftarrow \mathbf{v}_k - \lfloor \mu_{k,j} \rfloor \mathbf{v}_j;$

si *Condition de Lovász validée* **alors**

$k \leftarrow k + 1$

sinon

Échanger $\mathbf{v}_{k-1}$ et $\mathbf{v}_k;$

$k \leftarrow \max(k - 1, 2)$

Retourner $\{\mathbf{v}_1, \dots, \mathbf{v}_n\};$

Expliquons ce que fait l'algorithme LLL en mots pour clarifier le processus. Remarquons que notre pseudocode n'explicite pas les calculs des vecteurs de la base $\mathcal{B}^*$ de Gram-Schmidt. Ces derniers sont calculés dynamiquement lorsque nécessaire. Analysons la boucle *tant – que*, car

c'est le segment le plus important. Concentrons-nous sur la boucle *pour* imbriquée d'abord, et remarquons quelque chose d'intéressant.

Proposition 4. La condition de taille est vérifiée implicitement dans la boucle *pour*.

Démonstration. Posons à des fins d'analyse $\mathbf{v}'_k = \mathbf{v}_k - \lfloor \mu_{k,j} \rfloor \mathbf{v}_j$. Le coefficient $\mu'_{k,j}$, soit le coefficient calculé avec $\mathbf{v}'_k$ (donc après la mise à jour de $\mathbf{v}_k$) sera calculé comme

$$\mu'_{k,j} = \frac{|(\mathbf{v}_k - \lfloor \mu_{k,j} \rfloor \mathbf{v}_j) \cdot \mathbf{v}_j^*|}{\|\mathbf{v}_j^*\|^2}$$

ce qui, par linéarité du produit scalaire et par définition de Gram-Schmidt, donnerait

$$\mu'_{k,j} = \frac{|\mathbf{v}_k \cdot \mathbf{v}_j^*|}{\|\mathbf{v}_j^*\|^2} - \lfloor \mu_{k,j} \rfloor \frac{|\mathbf{v}_j \cdot \mathbf{v}_j^*|}{\|\mathbf{v}_j^*\|^2} = \mu_{k,j} - \lfloor \mu_{k,j} \rfloor$$

et par les propriétés de l'arrondissement vers un nombre entier, on a forcément que

$$|\mu'_{k,j}| \leq \frac{1}{2}.$$

□

Comme l'une des conditions est déjà validée, il ne reste qu'à tester la condition de Lovász pour s'assurer d'avoir obtenu une base LLL-réduite. Celle-ci est donc explicitement vérifiée : le vecteur passe le test, alors on peut itérer vers le prochain vecteur. Dans le cas contraire, il faut faire un pas en arrière et échanger $\mathbf{v}_{k-1}$ et $\mathbf{v}_k$ car les projections des vecteurs de la nouvelle base ne suivraient pas l'ordre quasi-croissant nécessaire à l'obtention d'une base utile. Tout compte fait, formalisons la puissance de l'algorithme que nous venons de décrire.

Théorème 4. Soit $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ une base d'un réseau L . Alors l'algorithme *LLL* ci-dessus retourne une base LLL-réduite de L en un nombre fini d'étapes car la base retournée satisfait les conditions de la définition 17. Avec $B = \max \|\mathbf{v}_i\|$, alors l'algorithme exécute la boucle tant-que $\mathcal{O}(n^2 \log n + n^2 \log B)$ fois. On remarque donc que *LLL* est un algorithme en temps polynomial.

Remarque 5. Il est intéressant d'observer que pour obtenir une base strictement parfaite, il aurait fallu utiliser la valeur 1 plutôt que la valeur $\frac{3}{4}$ dans la condition de Lovász. Cependant, c'est à ce jour un problème ouvert que de déterminer s'il est alors possible de résoudre le problème en temps polynomial, car il est possible pour l'algorithme d'entrer dans une boucle infinie d'échanges de vecteurs et de ne jamais s'arrêter.

3.4.3 Applications

Les applications de LLL se sont éparpillées dans plusieurs domaines. Ses motivations initiales en factorisation et en théorie des nombres sont toujours utilisées dans des logiciels de calcul formels. En cryptographie, LLL a permis de casser de nombreux systèmes de chiffrement auparavant jugés comme assez sûrs, tels que les problèmes de Knapsack et de RSA de bas niveau. Tel que mentionné plus tôt, LLL peut aussi résoudre les problèmes théoriques *apprSVP* et *apprCVP* lorsque leur relaxation est exponentielle.

À titre d'exemple, montrons comment LLL brisa le système GGH (on rappelle que GGH est un problème de type *CVP*). En 1987, Kannan [10] montra qu'il était possible de transformer une instance de *CVP* à n dimensions en un problème de *SVP* à $n + 1$ dimensions. Phong Nguyen montra donc qu'en utilisant cette technique et en exploitant une faiblesse de GGH, il pouvait transformer le *CVP* de GGH en un problème de *SVP* [19]. En effet, advenant sa résolution, le *SVP* lui donnait alors le vecteur d'erreur qui avait été injecté dans le système. Comme le vecteur d'erreur e est beaucoup plus petit que les autres vecteurs du réseau, on dit qu'il y a un «gap» entre ceux-ci, transformant le problème en instance *gapSVP*, que l'on définit comme suit :

Définition 18. (GapSVP_γ) Soit un réseau L de dimension n et un paramètre d de distance en entrée. Le problème GapSVP_γ , où γ est un paramètre d'approximation, cherche à trouver lequel de ces deux cas est validé :

1. Le plus court vecteur est très petit, soit $\lambda_1(\mathcal{L}) \leq d$.
2. Le plus court vecteur est grand, soit $\lambda_1(\mathcal{L}) > \gamma(n) \cdot d$.

De la manière dont le problème est construit, il est possible de garantir que l'un de ces deux cas est vrai. Le «gap» est donc l'espace dans lequel le plus court vecteur ne peut se trouver. Parfois, on notera $\text{GapSVP}_{\zeta, \gamma}$, où ζ est une borne supérieure sur la taille du plus petit vecteur du réseau L sur lequel se déroule le problème. Formellement, $\lambda_1(L) \leq \zeta$.

Dans ce cas-ci, comme le «gap» est plus grand que le facteur d'approximation de LLL, on peut garantir que LLL tombe forcément sur le vecteur le plus petit du réseau, soit le cas 1. Tout compte fait, LLL peut trouver facilement le vecteur d'erreur e dans GGH à haute dimension et déchiffrer très rapidement les messages envoyés.

Aujourd'hui, LLL sert plutôt de mesure de sécurité : lui et ses variants (tel que BKZ) simulent souvent la meilleure attaque possible contre un cryptosystème, et donc agissent comme gage de sécurité lorsqu'ils ne peuvent le casser. Son rôle a donc bien changé !

4 Le problème LWE

4.1 Définition

La sécurité d'un grand nombre de cryptosystèmes, tels que le mécanisme d'encapsulation de clé de Kyber et le schéma de signature de Dilithium, repose sur la difficulté de résoudre le problème de Learning With Errors (LWE). Notons que LWE est paramétrisé par 4 valeurs : deux entiers m et n tels que $m \gg n$, un module premier q , et une marge d'erreur B satisfaisant $B \ll q/2$.

Définition 19. Le problème Learning With Errors, que l'on notera $\text{LWE}(m, n, q, B)$, est défini comme suit. Choisissons une matrice $A \in_R \mathbb{Z}_q^{m \times n}$, un vecteur secret $\mathbf{s} \in_R \mathbb{Z}_q^n$ et un vecteur d'erreur $\mathbf{e} \in_R [-B, B]^m$. Posons ensuite $\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod{q} \in \mathbb{Z}_q^m$. Avec la paire $(A, \mathbf{b})$, on cherche à retrouver $\mathbf{s}$. On qualifiera m comme le nombre «d'échantillons» du problème LWE.

On remarque que si $B = 0$, alors $\mathbf{e} = 0$ et nous obtenons un système $A\mathbf{s} = \mathbf{b} \pmod{q}$ à résoudre pour $\mathbf{s}$. Un tel système se résolvant par élimination gaussienne sur $\mathbb{Z}_q$, le problème n'est pas difficile. LWE est difficile spécifiquement à cause du vecteur d'erreur $\mathbf{e}$, que l'on appellera aussi bruit du système. Par la définition du problème, on suppose implicitement que la solution est unique. Cependant, cela n'est qu'une garantie probabiliste, garantie qui tient de manière excessivement forte si le nombre de rangées m de A est beaucoup plus grand que le nombre de colonnes n . Comme ce sera le problème que nous étudierons le plus dans ce travail, il est pertinent de proposer une instance standard de ce dernier [15].

Exemple 5. Soit un problème $\text{LWE}(m, n, q, B)$ avec $m = 5, n = 3, q = 47, B = 2$. On génère donc la matrice A aléatoirement, tandis que $\mathbf{s}$ et $\mathbf{e}$ sont échantillonnés uniformément à partir de $\mathbb{Z}_q^n$ et $[-B, B]^m$ respectivement⁴. Après avoir calculé la valeur de $\mathbf{b}$, on divulgue le couple $(A, \mathbf{b})$ publiquement, ainsi que les valeurs propres au problème que l'on a posées plus tôt. On a donc

$$A = \begin{bmatrix} 27 & 13 & 13 \\ 1 & 46 & 23 \\ 16 & 13 & 30 \\ 18 & 16 & 19 \\ 22 & 0 & 3 \end{bmatrix} \in \mathbb{Z}_{47}^{5 \times 3}, \quad \mathbf{b} = \begin{bmatrix} 30 \\ 28 \\ 25 \\ 34 \\ 32 \end{bmatrix} \in \mathbb{Z}_{47}^5.$$

À travers cette instance précise, le but est donc de trouver le vecteur de solution $\mathbf{s} \in \mathbb{Z}_{47}^3$ et le vecteur de perturbation $\mathbf{e} \in [-2, 2]^5$ qui satisfont $A\mathbf{s} + \mathbf{e} = \mathbf{b} \pmod{47}$. On remarque qu'il y a trois solutions possibles : $\mathbf{s} = (2, 15, 12), \mathbf{e} = (1, 0, 2, 0, -1)$; $\mathbf{s} = (19, 34, 12), \mathbf{e} = (0, 2, 0, 1, 1)$; et $\mathbf{s} = (41, 33, 7), \mathbf{e} = (1, 0, -1, -2, 2)$.

4. Remarquons que cela n'est pas toujours le cas. Nombreuses sont les implémentations de LWE où $\mathbf{s}$ et $\mathbf{e}$ sont échantillonnés selon une distribution précise, telle que la gaussienne discrète ou bien la binomiale centrée.

Remarque 6. Bien que le système de l'exemple précédent ait plusieurs solutions, nous supposons que les problèmes sur lesquels nous travaillerons dans la suite de notre étude n'en ont qu'une.

Remarque 7. Il existe plusieurs variantes du problème LWE. Celui que nous avons présenté pour l'instant est le problème LWE de base, que l'on appelle aussi Search-LWE, comme on cherche à retrouver le vecteur $\mathbf{s}$. Un exemple d'alternative à ce dernier est le problème de LWE décisionnel (DLWE), où l'on fournit deux instances de LWE, une ayant une solution et l'autre n'en ayant pas, le but étant de trouver l'instance valide. Ces problèmes sont tous aussi difficiles algorithmiquement [22].

4.2 LWE et réseaux

Le problème de base ayant été défini, proposons maintenant le réseau découlant naturellement d'une instance de LWE. C'est le point tournant de notre travail : en montrant l'équivalence entre ces objets, nous ouvrons la porte à l'utilisation de techniques de résolution de problèmes sur les réseaux pour des instances de LWE.

Définition 20. Soit un problème $\text{LWE}(m, n, q, B)$ et son instance $(A, \mathbf{b})$ associée. Le *réseau LWE* associé à ceux-ci est défini par

$$L_A = \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{y} = A\mathbf{z} \pmod{q} \text{ pour un } \mathbf{z} \in \mathbb{Z}^n\}.$$

En mots, on peut voir ce réseau comme l'ensemble de tous les vecteurs $\mathbf{y}$ de dimension m que l'on peut générer depuis la multiplication entre la matrice A et un vecteur secret quelconque $\mathbf{z} \in \mathbb{Z}^n$, le tout étant pris $\pmod{q}$, mais sans ajouter le vecteur d'erreur $\mathbf{e}$. Sans le prouver, avançons le résultat suivant :

Théorème 5. Le réseau LWE $L_A \subseteq \mathbb{R}^m$ est un réseau d'entiers de plein-rang.

Ce constat nous permet de réutiliser tous les problèmes que nous avons vus depuis le début : en effet, comme le réseau L_A est de plein-rang, alors il satisfait le théorème de Minkowski. Cela nous permet d'obtenir les garanties nécessaires à nos problèmes de vecteurs sur des réseaux.

Considérons une instance de LWE $(A, \mathbf{b})$ telle que $\mathbf{b} = A\mathbf{s} + \mathbf{e}$ et définissons $\mathbf{y} = A\mathbf{s} \pmod{q} \in L_A$. En retrouvant $\mathbf{y}$ depuis $(A, \mathbf{b})$, il est alors facile de calculer le vecteur $\mathbf{s}$ à partir de $A\mathbf{s} = \mathbf{y} \pmod{q}$. On peut donc voir le problème LWE comme un problème de recherche de ce vecteur $\mathbf{y}$.

Comme le vecteur d'erreur $\mathbf{e}$ respecte $\mathbf{e} \in [-B, B]^m$, alors on a

$$\|\mathbf{b} - \mathbf{y}\|_2 = \|\mathbf{e}\|_2 \leq \sqrt{m}B.$$

En mots, on peut simplement remarquer que le vecteur $\mathbf{y}$ est forcément proche du vecteur cible

b. Observons donc une ressemblance entre LWE et le problème que l'on introduit ci-dessous.

Définition 21. Le problème de *décodage à distance bornée*, que l'on note BDD_α , est défini tel que l'on a $L \subseteq \mathbb{R}^m$ un réseau et $\mathbf{b} \in \mathbb{R}^m$ un vecteur avec la garantie qu'il existe un unique vecteur $\mathbf{y} \in L$ à distance α de $\mathbf{b}$. On cherche à trouver ce $\mathbf{y}$.

Pour s'assurer que le problème de BDD_α soit valide sur L_A et pour pouvoir affirmer que $\text{LWE} \leq \text{BDD}_\alpha$, il faut vérifier qu'il ne puisse pas exister deux vecteurs $\in L_A$ dans un rayon de α de $\mathbf{b}$. Soit $\mathbf{y}_1, \mathbf{y}_2$ les deux vecteurs les plus rapprochés de notre réseau et $\lambda_1(L_A) = \|\mathbf{y}_1 - \mathbf{y}_2\|_2$ la taille du plus petit vecteur de L_A que l'on nomme $\mathbf{y}_3$. On cherche à s'assurer que $2\alpha < \lambda_1(L_A)$. Si cette condition n'est pas vérifiée, comme L_A est fermé sous addition, alors il serait possible de trouver $\mathbf{y}_1$ et $\mathbf{y}_2 = \mathbf{y}_1 + \mathbf{y}_3$ dans un rayon α de $\mathbf{b}$: le problème BDD_α ne serait donc pas valide. Il faut alors absolument que $\alpha < \frac{\lambda_1(L_A)}{2}$.

Proposition 5. Constatons que $\text{LWE} \leq \text{BDD}_\alpha$ si ses paramètres sont instanciés pour pouvoir permettre $\alpha < \frac{\lambda_1(L_A)}{2}$, car cela garantit une solution unique. Depuis BDD , il est ensuite possible d'obtenir une seconde réduction, soit $\text{BDD} \leq \text{Unique-SVP}$, via la technique d'encastrement de Kannan [15].

Tout compte fait, remarquons que le problème LWE est donc visiblement équivalent à un problème sur les réseaux. Il est possible de transformer une instance de LWE en un réseau, et de poser un problème alternatif sur ce dernier. La solution proposée résoudra à la fois les deux problèmes.

4.3 SIS : un problème analogue

Avant de se lancer dans les preuves de sécurité et les techniques de déchiffrement concernant LWE , présentons le problème SIS , pour *Short Integer Solutions*, ou *Solutions Entières Courtes* en français. On considère le problème SIS comme le problème dual de LWE . Selon la technique de résolution utilisée, il peut souvent être pertinent de réduire LWE à SIS (ou vice-versa) pour résoudre une instance d'un des problèmes. Ces réductions sont souvent cependant conditionnelles : la réduction $\text{LWE} \leq \text{SIS}$ dépend des paramètres des problèmes, et la réduction $\text{SIS} \leq \text{LWE}$ est une réduction quantique⁵ [21]. Comme SIS a été proposé avant LWE , on dit souvent que le problème LWE est l'analogue facilitant le chiffrement au problème SIS .

Définition 22. Le problème $\text{SIS}(m, n, q, B)$ où q est un nombre premier, où m et n sont des entiers tels que $m \ll n$, et où B est une borne respectant $B \ll q/2$ et $B^n > q^m$ est défini comme le problème de recherche d'un vecteur non-nul $\mathbf{z} \in \mathbb{Z}^n$ tel que $A\mathbf{z} = 0 \pmod{q}$, avec $A \in_R \mathbb{Z}_q^{m \times n}$ une matrice aléatoire et $\mathbf{z} \in [-B, B]^n$.

5. Nous donnerons plus de détails par rapport à ces réductions dans la prochaine section

Remarque 8. Pour SIS, on cherche à déterminer une solution non-nulle dans le noyau de A (mod q). Bien que l'élimination gaussienne fonctionne efficacement pour cela, la solution doit ici être précisément une solution telle que ses coordonnées sont entre $[-B, B]$. C'est de là qu'apparaît la complexité.

Remarque 9. SIS se distingue de LWE par la dimension du vecteur que l'on recherche. En SIS, le système est sous-déterminé : il y a plus de variables que d'équations. En LWE, si l'on cherche strictement le vecteur $\mathbf{s}$, alors le système est surdéterminé : il y a plus d'équations que d'inconnues. Cela pourrait donc être impossible à résoudre : c'est là qu'intervient l'erreur $\mathbf{e}$, qui permet de déjouer la rigidité du système. Pour résoudre LWE, on cherchera donc souvent à trouver $\mathbf{e}$ plutôt que $\mathbf{s}$.

Définition 23. (Réseau représentatif de SIS) On associe une instance $\text{SIS}(m, n, q, B)$ au réseau défini par

$$\{L_A^\perp = \{z \in \mathbb{Z}^n : Az = 0 \pmod{q}\}.$$

On appellera ce réseau le *réseau représentatif* d'une instance SIS.

Afin d'expliciter le lien entre les problèmes SIS et LWE, proposons une manière de relier entre eux leurs réseaux sous-jacents. Débutons par un pré-requis mathématique

Définition 24. (Réseau dual) Le *réseau dual* L^* d'un réseau $L \subset \mathbb{R}^m$ est l'ensemble de tous les vecteurs d'un espace dont le produit scalaire avec n'importe quel vecteur de L donne un nombre entier. Formellement, on définit

$$L^* := \{\mathbf{w} \in \mathbb{R}^m : \langle \mathbf{w}, \mathbf{v} \rangle \in \mathbb{Z} \quad \forall \mathbf{v} \in L\}.$$

Proposition 6. Il est facile de passer du problème SIS au problème LWE. En effet, leurs réseaux sont duaux à un facteur q . On a donc

$$L_A = q \cdot (L_{A^\top}^\perp)^*.$$

À partir d'une base $\mathcal{B}$ de L_A du LWE, on peut retrouver une base $\mathcal{B}^*$ du réseau $L_{A^\top}^\perp$ de SIS en calculant $\mathcal{B}^* = q \cdot (\mathcal{B}^{-1})^T$ [22]. Cependant, notons que ce calcul détruit la «qualité» d'une base. Ce n'est pas parce qu'une base est bonne pour LWE qu'elle sera bonne lorsque transformée en base pour SIS.

On remarque ainsi que les deux problèmes sont intimement reliés. Leur différence principale réside dans leur nombre de solutions possibles : en effet, SIS admet quasiment toujours plusieurs solutions valides par design, tandis qu'un système LWE bien configuré n'en admettra qu'une. Cela permet au problème LWE de se transformer en un problème de décision, tandis que le problème

SIS n'a pas le choix de toujours être un problème de recherche.

Tout comme LWE était un problème que l'on remarquait être très similaire à BDD, il est possible de constater que SIS est analogue à SVP, comme dans les deux cas le but est de trouver un petit vecteur. Depuis l'article original de Miklós Ajtai [1], nombreux ont été les travaux entrepris essayant de quantifier la difficulté de SIS. Introduisons un nouveau problème qui nous servira de référence pour déterminer cette dernière.

Définition 25. (SIVP) Soit $\mathbf{B} \in \mathbb{Z}^{n \times n}$ la base d'un réseau $\mathcal{L}$. Le problème des *vecteurs indépendants les plus petits* est celui de la recherche des n vecteurs de réseau indépendants $\mathbf{S} = (\mathbf{s}_1, \dots, \mathbf{s}_n)$ (où $\mathbf{s}_i \in \mathcal{L}(\mathbf{B})$ pour tout i) tels que leur longueur est au maximum $\lambda_n(\mathcal{L})$.

Remarque 10. SIVP est très difficile, mais il est possible encore une fois de lui admettre une certaine relaxation (un facteur d'approximation). On notera alors $\text{SIVP}_{\gamma, n}$. Le problème n'est pas NP-Difficile pour les relaxations utilisées en cryptographie, mais n'affecte en rien sa sécurité [27]. Il est rare de quantifier les problèmes cryptographiques selon leur NP-Difficulté (qui quantifie les problèmes en leur pire cas) : on regarde plus souvent leur difficulté en *cas moyen*, et SIVP est un très bon candidat.

Sans surprise, avec ce que nous venons de présenter, nous arrivons au résultat suivant :

Théorème 6. Pour tout $m = \text{poly}(n)^6$, tout $B > 0$ et pour tout $q \geq B \cdot \text{poly}(n)$, résoudre $\text{SIS}(n, m, q, B)$, s'il est bien conditionné, est au moins aussi difficile que résoudre GapSVP_{γ} et SIVP_{γ} dans le pire cas, avec un $\gamma = B \cdot \text{poly}(n)$. On a donc $\text{GapSVP}_{\gamma} \leq \text{SIS}$.

En sachant que SIVP et GapSVP sont des problèmes très difficiles, il en découle que SIS l'est aussi : un algorithme résolvant efficacement SIS permettrait de résoudre efficacement SIVP et GapSVP. Pour l'instant, il n'en existe pas, et donc nous pouvons considérer que ce sont des problèmes ardues.

4.4 Sécurité

Avant d'argumenter sur la sécurité de LWE, introduisons une nouvelle quantité.

Définition 26. Le *taux d'erreur relatif*, noté $\alpha \in (0, 1)$, représente la proportion du bruit par rapport à la taille du module q . Formellement, on pose $\alpha = \frac{B}{q}$, ce qui implique que la borne d'erreur absolue devient $B = \alpha q$. Cette quantité permet de normaliser le bruit, ce qui aide à la formulation de preuves de sécurité asymptotiques.

Nombreuses sont les raisons de croire que le problème LWE est un problème difficile. Pour commencer, nous n'avons qu'à analyser les algorithmes d'état de l'art actuels pour LWE : ces derniers sont tous en temps exponentiel. D'autre part, la recherche actuelle a montré qu'il était

6. Où m peut être d'ordre $n, n^2, n^3, \dots$

possible de réduire LWE à des problèmes de réseaux. En 2009, Peikert [20] présente un résultat important, analogue à ce que nous avons posé pour SIS plus tôt.

Théorème 7. Soit un $m = \text{poly}(n)$, un taux d'erreur relatif $\alpha \in (0, 1)$ et un $q \geq \gamma \cdot \omega(\sqrt{\frac{\log n}{n}})^7$. Résoudre le problème $LWE(n, m, q, \alpha q)$ correspondant aux valeurs posées est aussi difficile que résoudre le pire cas du problème de $\text{GapSVP}_{\zeta, \gamma}$ ⁸ avec $\gamma \geq \frac{n}{\alpha \sqrt{\log n}}$ et $\zeta \geq \gamma$. On a donc $\text{GapSVP}_{\zeta, \gamma} \leq LWE$.

Pour les mêmes raisons que SIS , il est possible de déduire avec le théorème précédent que LWE est très probablement un problème difficile. Encore une fois, tout cela découle de la difficulté supposée des problèmes sur les réseaux.

Qu'en est-il cependant de la sécurité de LWE face aux avancées matérielles ? Ramenons-nous brièvement à ce que nous faisons au début, et terminons notre excursion dans le monde quantique. La cryptographie basée sur les réseaux a toujours eu comme l'une de ses motivations principales la création de protocoles de chiffrement résistants à l'ordinateur quantique. LWE n'est pas une exception : en effet, bien que nous ne l'ayons pas présenté initialement (pour ne pas voler le «punch» !), c'est Oded Regev qui, en 2005, proposa le problème LWE , et réduisit quantiquement le problème DGS (pour *Discrete Gaussian Sampling*) à LWE [25]. Par réduction quantique, nous entendons que s'il était possible de trouver un algorithme efficace pour LWE , alors il serait possible de trouver un algorithme quantique efficace pour DGS . Nous ne présenterons pas la preuve de sécurité de Regev, mais retenons-en quand même un corollaire important, que nous présentons ci-dessous.

Corollaire 1. Soit m, n, q des entiers et soit un taux d'erreur relatif $\alpha \in (0, 1)$ tel que $\alpha q > 2\sqrt{n}$. Alors s'il existe un algorithme efficace qui résout $LWE(m, n, q, \alpha q)$, alors il existe un algorithme quantique efficace qui résout le problème d'approximation de GapSVP , soit GapSVP_γ , à un facteur de $\tilde{O}(n/\alpha)$.

Démonstration. De manière très informelle : Regev a prouvé à la fois que $\text{GapSVP} \leq DGS$ classiquement et que $DGS \leq LWE$ quantiquement. On peut donc en déduire que $\text{GapSVP} \leq LWE$ quantiquement, que l'on viendra noter $\text{GapSVP} \leq_q LWE$. $\square$

Cette observation est particulièrement importante : en prouvant que GapSVP n'admet pas d'algorithme quantique efficace, nous prouverions par le fait même que les systèmes cryptographiques basés sur LWE sont sécuritaires face à la menace que pose l'ordinateur quantique. Pour y arriver, il est nécessaire d'introduire un dernier problème basé sur les réseaux.

Définition 27. (Unique-SVP_γ) Le problème d'approximation d'un unique-SVP est le problème

7. On utilise ω comme notation standard pour *strictement plus vite que*.

8. Revoir la définition 18 au besoin.

de recherche d'un plus petit vecteur isolé d'un réseau L à un facteur d'approximation γ . Notons qu'il a été prouvé que $\text{Unique-SVP}_\gamma \leq \text{GapSVP}_\gamma$.

Remarque 11. Avec $\text{Unique-SVP}_\gamma \leq \text{GapSVP}_\gamma$ et $\text{GapSVP}_\gamma \leq_q \text{LWE}$, on a $\text{Unique-SVP}_\gamma \leq_q \text{LWE}$.

Comme pour quasiment tous les problèmes modernes de cryptographie basés sur les réseaux, il n'existe pas d'algorithme classique efficace qui puisse résoudre Unique-SVP_γ . Dans le monde quantique, c'est la même chose : la meilleure tentative passerait via l'algorithme de Shor. Montrons donc pourquoi il échoue aussi (pour l'instant !). Rappelons que ce dernier permet des améliorations dans la résolution de problèmes quelconques seulement si ces derniers peuvent être réduits à des instances du *Hidden Subgroup Problem* (HSP) sur des groupes abéliens.

Le problème Unique-SVP_γ est particulièrement pertinent puisque trois ans avant son article sur LWE, Regev montre que ce dernier se réduit au problème de HSP sur le groupe diédral (donc que $\text{Unique-SVP}_\gamma \leq_q \text{HSP}_D$ [24]). Cependant, il y a ici un clash : le groupe diédral n'est pas abélien, et donc Shor ne fonctionne pas ! Il n'est donc pas possible de résoudre le problème de HSP ici afin de résoudre le problème Unique-SVP_γ . Puisque $\text{Unique-SVP}_\gamma \leq_q \text{LWE}$ et qu'à ce jour, il n'a pas été possible de résoudre efficacement Unique-SVP_γ , alors il est possible d'affirmer que le problème LWE est pour l'instant très sécuritaire.

Pour l'instant, il n'a jamais été possible de réduire les problèmes difficiles basés sur les réseaux à des groupes abéliens : cela explique pourquoi l'algorithme de Shor a toujours échoué sur ces derniers. Cependant, cela est plutôt une conjecture qu'une garantie. La sécurité post-quantique de LWE repose donc sur l'hypothèse voulant que la structure complexe des réseaux ne puisse jamais être transformée en groupe abélien.

4.5 Implémentation

Learning With Errors agit comme un problème pivot puisque nombreux sont les problèmes difficiles se réduisant à lui. Cela lui permet d'être polyvalent et d'admettre plusieurs variantes. La cryptographie basée sur le problème LWE classique tend à nécessiter des clés de grande taille, souvent d'ordre n^2 . D'un point de vue pratique, cela n'est pas efficace : il serait beaucoup plus favorable de réduire ces clés à un ordre linéaire. Souvent, les problèmes LWE sont implémentés sous des variantes algébriques. On peut par exemple penser à Ring-LWE et à Module-LWE. Dans ceux-ci, on remplace le groupe $\mathbb{Z}_q^n$ sur lequel on travaillait initialement par l'anneau $\mathbb{Z}_q[x]/\langle x^n + 1 \rangle$.

Sans entrer dans les détails d'algèbre, on peut voir cet anneau comme tous les polynômes possibles à coefficients dans $\mathbb{Z}_q$, restreints sous $x^n + 1$. Cette restriction agit un peu comme un opérateur modulo : lors du calcul d'un x^n , il faut appliquer l'égalité $x^n = -1$. Le degré est donc restreint à x^{n-1} . Ces structures permettent à la fois de diminuer la taille de l'encodage des valeurs

et d’accélérer les calculs effectués sur celles-ci, via la transformation de Fourier rapide [26]. Il est possible de prouver que ces problèmes sont aussi difficiles que le LWE classique.

C’est justement sur la variante MLWE que se base le cryptosystème Kyber, désormais standardisé sous le nom ML-KEM, pour *Module-Lattice-Based Key-Encapsulation-Mechanism*. C’est un système d’échange de clé : il est donc directement analogue au processus de Diffie-Hellman présenté plus tôt dans le travail. Le NIST, soit l’agence gouvernementale américaine responsable de l’adoption de standards en cryptographie, l’a choisi comme unique gagnant en 2024 pour le chiffrement général parmi 82 candidats [18]. Trois autres algorithmes ont aussi été retenus (dont deux sont aussi basés sur les réseaux !) mais ont des vocations différentes. Cependant, tous ont été sélectionnés pour une raison : ils sont efficaces et résistent à l’ordinateur quantique. Par cette standardisation, le NIST encourage les administrateurs de systèmes informatiques à débiter la transition vers ces nouveaux procédés.

5 Prérequis d’optimisation

Cette section vient conclure la section «revue de littérature» de ce rapport, mais est optionnelle au lecteur qui ne travaillera pas en optimisation. Elle nous est cependant nécessaire car dans la seconde partie du travail, le problème LWE sera réduit à un modèle d’optimisation quadratique. Afin de mieux comprendre et d’améliorer la performance de ce dernier, il est impératif d’en apprendre davantage sur la programmation en nombres entiers. Présentons donc à titre d’introduction la formalisation d’un problème d’optimisation en nombres entiers ainsi que les algorithmes les plus pertinents du domaine.

5.1 Fondements

Le problème LWE, transformé en problème d’optimisation, est difficile puisqu’il requiert la modélisation en problème de programmation en nombres entiers. Les problèmes d’optimisation linéaires classiques sont généralement assez faciles, puisqu’ils sont résolubles par l’algorithme du Simplexe : c’est travailler avec des entiers qui vient ajouter de la complexité. Débutons par poser des définitions qui seront assez utiles [33].

Remarque 12. Tel que mentionné plus haut, le modèle que nous présenterons plus tard est un modèle de programmation en nombres entiers quadratique, car la fonction objectif à minimiser est quadratique. Nous posons cependant nos définitions et nos prérequis généraux pour des problèmes de programmation linéaire dans un souci de simplicité et de standardisation. Cela n’a aucune incidence sur la justesse des propos, ces derniers s’appliquant autant au monde quadratique.

Définition 28. (Problème PLNE) Le problème linéaire de programmation en nombres entiers le

plus «pur», pour lequel nous utiliserons l'acronyme PLNE, est défini comme le suivant :

$$\max \{cx : Ax \leq b, x \in \mathbb{Z}_+^n\}.$$

Dans ce cas-ci, la fonction à maximiser est une fonction linéaire cx , sous les contraintes $Ax \leq b$, pour laquelle il faut s'assurer que les variables x soient entières et positives.

Note. De manière analogue, il est possible de poser le problème de programmation quadratique en nombres entiers, que l'on peut abrégé à PQNE, et le définir via l'expression suivante :

$$\min \{x^T Qx + cx : Ax \leq b, x \in \mathbb{Z}_+^n\}.$$

Définition 29. (Réalisation) On dira que l'ensemble $S = \{x \in \mathbb{Z}_+^n, Ax \leq b\}$ est la région réalisable d'un problème, qu'un $x \in S$ est une solution réalisable et qu'une instance est réalisable si $S \neq \emptyset$.

Définition 30. (Optimalité) On dira qu'un point x^* est une solution *optimale* si pour tout $x \in S$, $cx^* \geq cx$. Parfois, les problèmes de PLNE n'ont pas de solution optimale. Dans un tel cas, nous dirons qu'une instance est *non-bornée*. Il est possible de prouver qu'une instance de PLNE admet soit une solution optimale, soit elle est non-bornée.

Généralement, les solutions possibles à un problème de PNE sont décrites implicitement. Par exemple, on pourrait définir un problème où l'on cherche à acheter des articles qui coûtent respectivement 2 et 3\$, avec un budget maximal de 10\$. Si x^1 et x^2 représentent la quantité achetée de chaque article, on pourrait poser les multiples choix possibles via l'ensemble $S = \{(x^1, x^2) \in \mathbb{Z}_+^2 : 2x^1 + 3x^2 \leq 10\}$. L'un de nos objectifs sera alors de trouver une description de cet ensemble via des inégalités linéaires (que l'on appellera des facettes), transformant un problème implicite en un problème explicite, équivalent et continu. Advenant un succès absolu, il serait alors possible d'appliquer l'algorithme du Simplexe pour résoudre le problème rapidement.

Définition 31. (Enveloppe convexe) Soit un ensemble $S \subseteq \mathbb{R}^n$. Un point $x \in \mathbb{R}^n$ est une combinaison convexe de points de S s'il existe un ensemble de points finis $\{x^i\}_{i=1}^t$ dans S et un $\lambda \in \mathbb{R}_+^t$ tels que $\sum_{i=1}^t \lambda_i = 1$ et $x = \sum_{i=1}^t \lambda_i x^i$. L'*enveloppe convexe* de S , que l'on note $\text{conv}(S)$, est l'ensemble de tous les points qui sont des combinaisons convexes de points dans S . Graphiquement, cela se visualise comme la figure ci-dessous, où l'enveloppe convexe des points noirs est la zone ombragée en bleue.

Définition 32. (Polyèdre) Un *polyèdre* $P \subseteq \mathbb{R}^n$ est un ensemble de points satisfaisant un nombre fini d'inégalités linéaires. Formellement, on a $P = \{x \in \mathbb{R}^n : Ax \leq \mathbf{b}\}$ où la matrice augmentée $[A \mid \mathbf{b}]$ est de taille $m \times (n + 1)$.

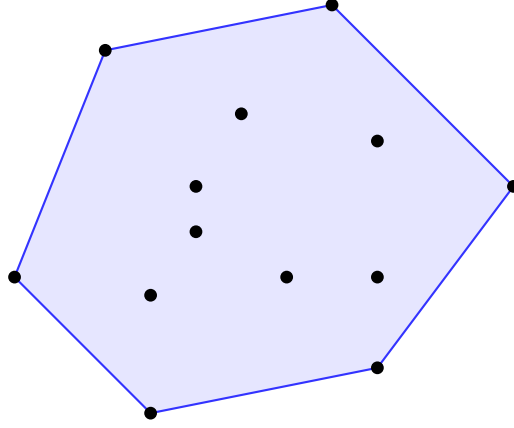


FIGURE 3 – Enveloppe convexe ($\text{conv}(S)$) d'un ensemble de points S .

Compte tenu de ces définitions, présentons maintenant les deux principaux algorithmes de résolution de problèmes de programmation en nombres entiers.

5.2 Algorithmes principaux

5.2.1 Branch-and-Bound

À travers nos futurs tests d'évaluation de performance, nous utiliserons souvent le terme *noeuds* pour quantifier la vitesse d'un solveur de programmation en nombres entiers. Ce terme provient de l'algorithme *Branch-and-Bound* (BB), et donc il convient de le présenter ici. La théorie vient ici du livre *Integer Programming* de Conforti et al. [4]. Pour commencer, rappelons-nous simplement que nous travaillons avec un problème de PLNE tel que

$$\max \{cx : x \in S\},$$

où S est la région réalisable du problème.

Définition 33. (Relaxation continue) La *relaxation continue* d'un problème de PLNE tel que présenté plus haut est définie par le problème d'optimisation s'exprimant par

$$\max \{cx : Ax \leq b, x \in \mathbb{R}_+^n\}.$$

Nous dénoterons la solution de cette relaxation sous x^0 et la valeur optimale du problème comme étant $z^0 = cx^0$. Le polyèdre correspondant à cette relaxation sera défini via $P_0 := \{x \in \mathbb{R}_+^n : Ax \leq b\}$. Remarquons que la région réalisable d'un problème de PLNE est un sous-ensemble du polyèdre correspondant à sa relaxation. Formellement, on a $S \subseteq P_0$. Si z^* est la solution entière au problème de base, on a alors forcément $z^* \leq z^0$. Dans le cas où le vecteur $x^0 \in S$, on a $z^* = z^0$, et le problème est alors résolu.

Supposons cependant le cas contraire : il existe une composante x_j^0 fractionnaire. On appelle alors l'algorithme BB : il divise alors S en deux sous-espaces disjoints définis par

$$S_1 := S \cap \{x \in \mathbb{R}_+^n : x_j \leq \lfloor x_j^0 \rfloor\}, \quad S_2 := S \cap \{x \in \mathbb{R}_+^n : x_j \geq \lceil x_j^0 \rceil\}.$$

Toute solution entière respecte forcément l'une de ces deux contraintes : il est donc possible d'affirmer que (S_1, S_2) est une partition de S . Nous venons donc de créer deux nouveaux sous-problèmes d'optimisation, définis par

$$\text{PLNE}_1 : \max\{cx : x \in S_1\}, \quad \text{PLNE}_2 : \max\{cx : x \in S_2\},$$

le terme *noeud* désignant justement l'un de ces sous-problèmes. La solution optimale au problème PLNE de base sera alors forcément la meilleure solution retournée par PLNE_1 et PLNE_2 . On remarque qu'il y a donc naturellement un potentiel à la construction d'une structure récursive. Considérons les relaxations linéaires des problèmes PLNE_1 et PLNE_2 , que l'on notera LP_1 et LP_2 , associées aux polyèdres P_1, P_2 , relaxés de manière analogue à P_0 . Analysons les avenues pouvant découler de ces problèmes relaxés.

1. Si l'un des LP_i est *infaisable*, c'est-à-dire que $P_i = \emptyset$, alors $S_i = \emptyset$. On arrête donc d'évaluer cette branche. On dira qu'elle est **élaguée par infaisabilité**.
2. Si la solution optimale x_i à un LP_i associée à la valeur z_i est une solution entière, alors la solution x^i est une solution à PLNE_i et donc est candidate au problème de PLNE original. Comme $S_i \subseteq S$, on a que $z_i \leq z^*$. On arrête donc d'évaluer cette branche, et on dira qu'elle est **élaguée par intégrité**.
3. Si x^i contient des composantes fractionnaires mais que sa valeur optimale relaxée satisfait $z_i \leq \bar{z}$, où $\bar{z}$ est la meilleure solution actuelle, alors le sous-ensemble S_i ne peut pas contenir de solution entière strictement meilleure que celle déjà connue. On arrête donc d'évaluer cette branche, on dira qu'elle est **élaguée par borne**.
4. Si x^i n'est pas un vecteur d'entier mais que z_i respecte $z_i > \bar{z}$, alors S_i peut toujours contenir une solution optimale au problème de PLNE original. Soit $x_{j'}^i$ une composante fractionnaire de x^i . On peut alors diviser S_i de manière identique à ce que l'on a fait plus tôt avec S , et on obtient :

$$S_{i_1} := S_i \cap \{x \in \mathbb{R}_+^n : x_{j'} \leq \lfloor x_{j'}^i \rfloor\}, \quad S_{i_2} := S_i \cap \{x \in \mathbb{R}_+^n : x_{j'} \geq \lceil x_{j'}^i \rceil\}.$$

Remarquons que c'est le point 4 qui génère l'arborescence : depuis ce dernier, il est possible de réappliquer le processus récursivement. À un certain moment, tous les noeuds ouverts seront épuisés, l'algorithme ayant terminé son exécution et ayant trouvé la meilleure solution entière.

Remarque 13. La détermination d’une solution à une relaxation continue quelconque se calcule de manière efficace en pratique : seul l’algorithme du Simplexe est nécessaire. La qualité de la relaxation continue initiale détermine donc directement le nombre de noeuds que prendra une exécution de l’algorithme Branch-and-Bound. Plus la relaxation continue initiale modélise bien le problème discret auquel elle correspond, plus il sera rapide pour l’algorithme Branch-and-Bound de déterminer la solution optimale à un problème de PLNE.

5.2.2 Méthode des plans sécants

Contrairement à BB, la méthode des plans sécants vient directement resserrer la relaxation continue initiale sans jamais retirer de points entiers de S . Reprenons le problème de PLNE de base, soit $\max \{cx : x \in S\}$, ainsi que sa relaxation continue $\max \{cx : x \in P_0\}$. Supposons que la solution optimale x^0 de la relaxation continue contient au minimum une composante fractionnaire (nous serions donc dans les cas 3 et 4 de BB).

Définition 34. (Inégalité valide) On dira qu’une inégalité $\alpha x \leq \beta$ est *valide* pour un ensemble $K \subseteq \mathbb{R}^d$ si elle est satisfaite pour tous les points $\bar{x} \in K$.

Définition 35. (Plan sécant) Un *plan sécant* (ou une *coupe*) pour une solution fractionnaire x^0 est une inégalité $\alpha x \leq \beta$ valide pour tous les points de S mais qui est violée par x^0 , la solution fractionnaire courante.

Il existe forcément un plan sécant lorsque x^0 est une solution retournée par l’algorithme du Simplexe : la preuve est en [4]. Par cette garantie d’existence, on peut poser $\alpha x \leq \beta$ un tel plan sécant, ce qui nous permet de définir $P_1 := P_0 \cap \{x : \alpha x \leq \beta\}$. On a alors $S \subseteq P_1 \subset P_0$. Nous pouvons donc affirmer que la relaxation P_1 est plus forte que la relaxation P_0 initiale, l’espace de recherche continu ayant diminué. L’algorithme des plans sécants se base donc sur cette approche de manière récursive : il cherche à rapprocher progressivement les P_i vers l’enveloppe convexe de S . Dans un tel cas, il serait facile pour le Simplexe de trouver une solution entière optimale.

En pratique, les solveurs modernes combinent les plans sécants à l’algorithme Branch-and-Bound. On appelle cette méthode hybride Branch-and-Cut, et c’est ce que fait Gurobi : il applique d’abord des coupes avant de faire un branchement, ce qui diminue la taille de l’arbre final.

PARTIE 2

Modélisation et expérimentations

6 Réduction principale : Search-LWE $\leq$ PQNE

6.1 Idée générale

L'inspiration derrière le croisement entre le problème LWE et la programmation en nombres entiers fut présentée par Masaaki Shirase [28] : dans une prépublication de 2023, il réduit le problème Search-LWE, soit le problème LWE classique, à un problème de programmation en nombres entiers. Exhibons pour commencer cette réduction car elle sera centrale aux développements de ce travail.

Rappel. Le problème Search-LWE, soit la forme la plus basique de LWE, pose la relation suivante

$$As + \mathbf{e} \equiv \mathbf{b} \pmod{q}$$

depuis les valeurs $A \in \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \in \mathbb{Z}_q^n$, $\mathbf{e} \in \mathbb{Z}^m$ et $\mathbf{b} \in \mathbb{Z}_q^m$. Dans le cas précis de la réduction, nous travaillerons avec un vecteur $\mathbf{e}$ tel que ses composantes sont choisies selon une loi $N(0, \sigma^2)$, soit une normale centrée sur 0 avec écart-type σ . On rappelle que le but de cette construction, et donc le but de LWE, est de déterminer $\mathbf{s}$ depuis le couple $(A, \mathbf{b})$.

Depuis un problème LWE aux valeurs telles que posées plus haut, argumentons maintenant qu'il est possible de construire un problème de programmation en nombres entiers tel que sa solution optimale ou sa solution provisoire, toutes deux devant être des vecteurs à petite norme, sont égales à $\mathbf{e}$, et ce à haute probabilité. Notons qu'il est possible de facilement déterminer la solution $\mathbf{s}$ depuis un sous-ensemble des composantes de $\mathbf{e}$: nous expliciterons ce lien plus tard.

Remarque 14. Plus tard, nous observerons qu'il existe une multitude de couples $(\hat{\mathbf{s}}, \hat{\mathbf{e}})$ valides, soit des couples (secret, erreur) qui correspondent à une instance $(A, \mathbf{b})$ de LWE. Notre modèle de PNE doit donc trouver parmi ces options l'unique couple $(\mathbf{s}, \mathbf{e})$ correspondant au secret recherché, et c'est là que la construction du problème LWE entre en jeu. Quand les composantes de $\mathbf{e}$ sont choisies selon une $N(0, \sigma^2)$, la probabilité que $\|\mathbf{e}\|$ soit grande est très faible. À très forte probabilité, il est possible d'affirmer que le vecteur d'erreur que l'on recherche est le vecteur d'erreur à la plus petite norme parmi tous les vecteurs d'erreurs valides. On voudra donc minimiser la norme de ce dernier dans notre programmation en nombres entiers. De manière analogue, la probabilité qu'il existe plus d'une solution à une instance particulière de LWE lorsque $\mathbf{e}$ est échantillonné selon $N(0, \sigma^2)$ est très faible. Ces deux quasi-garanties nous permettent d'entreprendre notre travail sans nous soucier d'avoir un problème mal initialisé, et nous assurent qu'il existe une direction de recherche.

6.2 Partitionnement des matrices et des vecteurs

Pour commencer, redéfinissons $A, \mathbf{b}, \mathbf{e}$ en introduisant de nouvelles variables $A_0, A_1, \mathbf{b}_0, \mathbf{b}_1, \mathbf{e}_0, \mathbf{e}_1$.

$$A = \begin{pmatrix} A_0 \\ A_1 \end{pmatrix}, \quad \text{où} \begin{cases} A_0 \text{ est la matrice } n \times n \text{ supérieure de } A, \\ A_1 \text{ est la matrice } (m - n) \times n \text{ inférieure de } A. \end{cases}$$

$$\mathbf{b} = \begin{pmatrix} \mathbf{b}_0 \\ \mathbf{b}_1 \end{pmatrix}, \quad \text{où} \begin{cases} \mathbf{b}_0 \text{ est le vecteur colonne de dimension } n \text{ supérieur de } \mathbf{b}, \\ \mathbf{b}_1 \text{ est le vecteur colonne de dimension } m - n \text{ inférieur de } \mathbf{b}. \end{cases}$$

$$\mathbf{e} = \begin{pmatrix} \mathbf{e}_0 \\ \mathbf{e}_1 \end{pmatrix}, \quad \text{où} \begin{cases} \mathbf{e}_0 \text{ est le vecteur d'erreur de dimension } n \text{ supérieur de } \mathbf{e}, \\ \mathbf{e}_1 \text{ est le vecteur d'erreur de dimension } m - n \text{ inférieur de } \mathbf{e}. \end{cases}$$

Exemple 6. Tiré directement de l'article, posons l'exemple suivant.

$$\text{Pour la matrice } A = \begin{pmatrix} 0 & 5 \\ 1 & 6 \\ 2 & 7 \\ 3 & 8 \\ 4 & 9 \end{pmatrix}, \quad \text{on a } A_0 = \begin{pmatrix} 0 & 5 \\ 1 & 6 \end{pmatrix} \text{ et } A_1 = \begin{pmatrix} 2 & 7 \\ 3 & 8 \\ 4 & 9 \end{pmatrix}.$$

$$\text{Pour le vecteur } \mathbf{b} = \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}, \quad \text{on a } \mathbf{b}_0 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \text{ et } \mathbf{b}_1 = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}.$$

Remarquons alors que l'équation globale de l'instance de LWE peut se réécrire sous la forme du système suivant :

$$A_0 \mathbf{s} + \mathbf{e}_0 \equiv \mathbf{b}_0 \pmod{q} \tag{1}$$

$$A_1 \mathbf{s} + \mathbf{e}_1 \equiv \mathbf{b}_1 \pmod{q}. \tag{2}$$

Note. Pour le reste de notre étude, nous simplifierons la notation utilisée. Lorsque nous travaillons en $(\text{mod } q)$, on abrégera $a \equiv b \pmod{q}$ à $a \equiv b$. Pour deux vecteurs $\mathbf{v} = (v_1, v_2, \dots, v_n)$ et $\mathbf{w} = (w_1, w_2, \dots, w_n)$, on notera $\mathbf{v} \equiv \mathbf{w}$ si $v_i \equiv w_i \ \forall i$. Pour deux matrices $A = (a_{i,j})$ et $B = (b_{i,j})$, on notera $A \equiv B$ si $a_{i,j} \equiv b_{i,j} \ \forall i, j$.

6.3 Applications linéaires

Nous proposerons dans cette section des applications affines aux propriétés intéressantes nous permettant de transformer le problème LWE initial davantage. Débutons par une définition, liée directement à ces dernières.

Définition 36. (Matrice régulière sur un corps) On dit qu'une matrice $A \in \mathbb{Z}_q^{m \times n}$ est régulière sur $\mathbb{Z}_q$ si et seulement si $\det(A) \not\equiv 0 \pmod{q}$.

Proposition 7. Supposons que $A_0 \in \mathbb{Z}_q^{n \times n}$ est régulière sur $\mathbb{Z}_q$. Alors, il existe une matrice $A_0^{-1} \in \mathbb{Z}_q^{n \times n}$ qui respecte $A_0 A_0^{-1} \equiv A_0^{-1} A_0 \equiv I_n$. Simplement dit, A_0^{-1} est l'inverse de A_0 sur le corps $\mathbb{Z}_q$. Avec $A_0^{-1}, A_1, \mathbf{b}_0, \mathbf{b}_1$, définissons l'application affine suivante :

$$\begin{aligned} \phi : \mathbb{R}^n &\rightarrow \mathbb{R}^{m-n} \\ \mathbf{v} &\rightarrow A_1 A_0^{-1} \mathbf{v} + \mathbf{b}_1 - A_1 A_0^{-1} \mathbf{b}_0. \end{aligned}$$

Avec cette application ϕ , définissons-en une autre :

$$\begin{aligned} \psi : \mathbb{R}^n &\rightarrow \mathbb{R}^{m-n} \\ \mathbf{v} &\rightarrow \phi(\mathbf{v}) - \phi(\mathbf{0}_n), \end{aligned}$$

où l'on définit $\mathbf{0}_n$ comme le vecteur colonne nul à dimension n .

Présentons maintenant les résultats découlant de ces applications. Seule la dernière preuve sera présentée, les autres étant disponibles dans l'article de Shirase. Elles sont toutes calculatoires, et prennent moins de cinq lignes.

1. $\phi(\mathbf{e}_0) \equiv \mathbf{e}_1$. On remarque qu'il est possible de déduire la seconde partie du vecteur d'erreur en appliquant ϕ à la première partie de ce dernier.
2. Supposons que nous avons $\mathbf{v}_0 \in \mathbb{R}^n$ et $\mathbf{v}_1 \in \mathbb{R}^{m-n}$ tels que $\phi(\mathbf{v}_0) \equiv \mathbf{v}_1$. Soit $\hat{\mathbf{s}} \equiv A_0^{-1}(\mathbf{b}_0 - \mathbf{v}_0)$. Alors l'équation

$$\begin{pmatrix} A_0 \\ A_1 \end{pmatrix} \hat{\mathbf{s}} + \begin{pmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \end{pmatrix} = \begin{pmatrix} \mathbf{b}_0 \\ \mathbf{b}_1 \end{pmatrix}$$

est valide. Via cette propriété, on remarque que dès que la condition 1 est validée, alors il est possible de générer une solution candidate exacte. On peut le visualiser comme si l'on travaillait toujours avec un couple $(\mathbf{v}, \hat{\mathbf{s}})$: cependant, $\hat{\mathbf{s}}$ est dépendant de $\mathbf{v}$, et donc on a réduit le problème à la recherche d'un $\mathbf{v}$ idéal.

3. $\phi(\mathbf{0}_n) = \mathbf{b}_1 - A_1 A_0^{-1} \mathbf{b}_0$. Cette propriété vient confirmer que la transformation ϕ n'est pas

totalelement linéaire, car il peut exister une ordonnée à l'origine.

4. $\psi(\mathbf{v}) = A_1 A_0^{-1} \mathbf{v}$. Contrairement à ϕ , ψ est une application linéaire.
5. $\sum k_i (\phi(\mathbf{v}_i) - \phi(\mathbf{0}_n)) = \phi(\sum k_i \mathbf{v}_i) - \phi(\mathbf{0}_n)$ pour $k_i \in \mathbb{Z}$. Via cette presque-linéarité, on remarque que même si ϕ n'est pas linéaire, on peut quand même manipuler des combinaisons linéaires de vecteurs sous cette application via ψ . La preuve ci-bas explicite particulièrement bien cette relation :

Démonstration.

$$\begin{aligned} \sum k_i (\phi(\mathbf{v}_i) - \phi(\mathbf{0}_n)) &= \sum k_i \psi(\mathbf{v}_i) \\ &= \psi \left(\sum k_i \mathbf{v}_i \right) \\ &= \phi \left(\sum k_i \mathbf{v}_i \right) - \phi(\mathbf{0}_n). \end{aligned}$$

□

Remarque 15. Depuis une instance $(A, \mathbf{b})$ de LWE, en posant $A_0, \mathbf{b}_0, \mathbf{e}_0$ tels que définis en 6.2 et en supposant que A_0 est régulière sur $\mathbb{Z}_q$, on a alors $\mathbf{s} = A_0^{-1}(\mathbf{b}_0 - \mathbf{e}_0) \bmod q$. Il est donc facile de calculer $\mathbf{s}$ depuis $\mathbf{e}_0$, ce que nous avons avancé plus tôt. Cela n'aurait pas été le cas si nous avions utilisé l'entièreté des composantes $A, \mathbf{b}, \mathbf{e}$: il n'aurait pas été possible d'inverser la matrice A car elle n'aurait pas été carrée.

6.4 Construction du problème

Proposons successivement les équations qui nous permettront d'obtenir le problème de programmation en nombres entiers représentatif du problème LWE. Pour commencer, si on pose $\mathbf{e} = (e_1, e_2, \dots, e_m)$, alors il est possible d'écrire $\mathbf{e}_0$ et $\mathbf{e}_1$ tels que

$$\begin{cases} \mathbf{e}_0 = (e_1, e_2, \dots, e_n)^T \\ \mathbf{e}_1 = (e_{n+1}, e_{n+2}, \dots, e_m)^T. \end{cases} \quad (3)$$

Avec un ensemble $\{\boldsymbol{\epsilon}_1, \dots, \boldsymbol{\epsilon}_n\}$ de vecteurs unitaires de dimension n tels que la i^{eme} composante d'un $\boldsymbol{\epsilon}_i$ est de valeur 1 tandis que le reste est nul⁹, il est possible d'écrire $\mathbf{e}_0$ comme

$$\mathbf{e}_0 = e_1 \boldsymbol{\epsilon}_1 + e_2 \boldsymbol{\epsilon}_2 + e_3 \boldsymbol{\epsilon}_3 + \dots + e_n \boldsymbol{\epsilon}_n. \quad (4)$$

9. Par exemple, $\boldsymbol{\epsilon}_2 = (0, 1, 0, 0, \dots, 0)$

Pour $j = 1, 2, \dots, n$, on pose

$$\mathbf{w}_j = \psi(\epsilon_j) = \phi(\epsilon_j) - \phi(\mathbf{0}_n) \in \mathbb{Z}_q^{m-n}, \quad (5)$$

qu'il sera possible de décomposer en

$$\mathbf{w}_j = (w_{1,j}, w_{2,j}, \dots, w_{m-n,j})^T, \text{ avec } w_{i,j} \in \mathbb{Z}_q. \quad (6)$$

On pose aussi

$$\phi(\mathbf{0}_n) = (u_1, u_2, \dots, u_{m-n})^\top, \text{ avec } u_i \in \mathbb{Z}_q. \quad (7)$$

Il sera alors possible de calculer :¹⁰

$$\mathbf{e}_1 \equiv \phi(\mathbf{e}_0) \quad (8)$$

$$= e_1 \mathbf{w}_1 + \dots + e_n \mathbf{w}_n + \phi(\mathbf{0}_n). \quad (9)$$

En isolant une ligne du système, on peut retrouver, avec (3) et (7)

$$e_{n+i} \equiv w_{i,1}e_1 + w_{i,2}e_2 + \dots + w_{i,n}e_n + u_i \quad (10)$$

pour $i = 1, 2, \dots, m - n$. On peut alors poser le système d'équations linéaire suivant sur $\mathbb{Z}_q$:

$$\begin{cases} w_{1,1}x_1 + w_{1,2}x_2 + \dots + w_{1,n}x_n + u_1 \equiv x_{n+1} \\ w_{2,1}x_1 + w_{2,2}x_2 + \dots + w_{2,n}x_n + u_2 \equiv x_{n+2} \\ \vdots \\ w_{m-n,1}x_1 + w_{m-n,2}x_2 + \dots + w_{m-n,n}x_n + u_{m-n} \equiv x_m. \end{cases} \quad (11)$$

On vient donc de créer un système où les variables $x_1, x_2, \dots, x_m$ ont une solution $(x_1, x_2, \dots, x_m) = (e_1, e_2, \dots, e_m)$. Cependant, comme le nombre de variables est plus grand que le nombre d'équations, la solution n'est pas unique. Modifions donc progressivement la modélisation afin de remédier à ce problème. Il est possible de modifier l'équation (10) en transformant le modulo en additionnant un multiple de q . On obtient donc

$$w_{i,1}e_1 + w_{i,2}e_2 + \dots + w_{i,n}e_n - e_{n+i} + qf_i = -u_i \quad (12)$$

avec un $f_i \in \mathbb{Z}$.

Réduisons maintenant l'espace de recherche en appliquant des bornes sur f_i et sur les compo-

10. Les détails du calcul sont dans l'article.

santes e_i de $\mathbf{e}$. Pour commencer, il est évident qu'il est possible de fixer un $t \in \mathbb{N}$ tel que

$$-t \leq e_i \leq t$$

avec une très grande probabilité comme les e_i , avec $i = 1, 2, \dots, m$, sont échantillonnés selon une loi normale centrée autour de 0. On peut considérer ce t comme un paramètre ajustable. De plus, avec l'équation (12) et comme $0 \leq w_{i,j} \leq q-1$, on peut poser les bornes suivantes sur f_i

$$-\left\lfloor \frac{t(nq - n + 1) + q - 1}{q} \right\rfloor \leq f_i \leq \left\lfloor \frac{t(nq - n + 1)}{q} \right\rfloor \quad (13)$$

avec $i = 1, 2, \dots, m - n$.

Il ne nous reste alors qu'à prouver qu'il est possible de calculer efficacement les valeurs de $w_{i,j}$ et de u_i et nous pourrions poser le problème de programmation en nombres entiers en son entièreté.

Lemme 1. Soit $(A, \mathbf{b})$ une instance LWE, où $A \in \mathbb{Z}_q^{m \times n}$ et $\mathbf{b} \in \mathbb{Z}_q^m$. On pose A_0 et t_0 tel que l'on avait fait dans la section 6.2. Supposons que A_0 est régulière sur $\mathbb{Z}_q$. Alors :

1. Il est possible de poser une matrice W de taille $(m - n) \times n$ où $W = (w_{i,j})$ en utilisant les $w_{i,j}$ de l'équation (6). Pour la calculer, on a $W \equiv A_1 A_0^{-1}$. Il s'agit d'un calcul rapide.
2. Il est possible de calculer le vecteur entier de u_i tel que donné par l'équation (7) via $(u_1, u_2, \dots, u_{m-n})^T \equiv \mathbf{b}_1 - A_1 A_0^{-1} \mathbf{b}_0$. Il s'agit aussi d'un calcul rapide.

6.5 Modèle complet

Théorème 8. (Réduction de LWE à PNE)

Soit $(A, \mathbf{b})$ une instance LWE, où $A \in \mathbb{Z}_q^{m \times n}$ avec $n < m$, et $\mathbf{b} \in \mathbb{Z}_q^m$. On pose A_0 et t_0 tel que l'on avait fait dans la section 6.2 et on suppose que A_0 est régulière sur $\mathbb{Z}_q$. Les variables $w_{i,j}$ et u_i avec $i = 1, \dots, m - n$; $j = 1, \dots, n$ sont calculées telles que dans le lemme précédent, et une valeur pour le paramètre $t \in \mathbb{N}$ est choisie. Il est alors possible de construire le problème de programmation en nombres entiers suivant :

$$\begin{aligned} \text{minimiser : } & \sum_{k=1}^m x_k^2 \\ \text{sujet à : } & \sum_{j=1}^n w_{i,j} x_j - x_{n+i} + q f_i = -u_i, & \forall i \in \{1, 2, \dots, m - n\} \\ & -t \leq x_k \leq t, & \forall k \in \{1, 2, \dots, m\} \\ & -\left\lfloor \frac{t(nq - n + 1) + q - 1}{q} \right\rfloor \leq f_i \leq \left\lfloor \frac{t(nq - n + 1)}{q} \right\rfloor, & \forall i \in \{1, 2, \dots, m - n\} \\ & x_k, f_i \in \mathbb{Z} \end{aligned}$$

Depuis une solution $(x_1, x_2, \dots, x_m)^T = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_m)$ optimale ou provisoire à petite norme à ce problème, il est alors possible de poser $\hat{\mathbf{x}}_0 = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n) \in \mathbb{Z}_q^n$. Depuis ce dernier, on peut retrouver $\hat{\mathbf{s}} = A_0^{-1}(\mathbf{b}_0 - \hat{\mathbf{x}}_0) \bmod q$, ce qui est la solution à l'instance $\text{LWE}(A, \mathbf{b})$ à très haute probabilité.

Démonstration. Posons l'estimateur de l'erreur $\hat{\mathbf{x}} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_m)^T \in \mathbb{Z}^m$ et sa seconde partie $\hat{\mathbf{x}}_1 = (\hat{x}_{n+1}, \hat{x}_{n+2}, \dots, \hat{x}_m)^T \in \mathbb{Z}^{m-n}$. D'après la construction du problème, la relation $\hat{\mathbf{x}}_1 \equiv \phi(\hat{\mathbf{x}}_0)$ est respectée. Nous avons alors, via le deuxième résultat découlant de nos applications linéaires :

$$\mathbf{b} = A\hat{\mathbf{s}} + \hat{\mathbf{x}} \implies \mathbf{b}_0 = A_0\hat{\mathbf{s}} + \hat{\mathbf{x}}_0 \quad (14)$$

La véritable solution $\mathbf{s}$ de l'instance $(A, \mathbf{b})$ satisfait la relation globale :

$$\mathbf{b} = A\mathbf{s} + \mathbf{e} \implies \mathbf{b}_0 = A_0\mathbf{s} + \mathbf{e}_0 \quad (15)$$

Si la norme $\|\hat{\mathbf{x}}\|$ est suffisamment petite, on en déduit que $\hat{\mathbf{x}} = \mathbf{e}$, ce qui implique que $\hat{\mathbf{x}}_0 = \mathbf{e}_0$ avec une haute probabilité. Dans ce cas, il suffit donc d'égaliser les $\mathbf{b}_0$, et on obtient :

$$A_0\hat{\mathbf{s}} \equiv A_0\mathbf{s} \quad (16)$$

$$\hat{\mathbf{s}} \equiv A_0^{-1}A_0\mathbf{s} \equiv \mathbf{s} \quad (17)$$

Par conséquent, $\hat{\mathbf{s}} = \mathbf{s}$ puisque $\mathbf{s}, \hat{\mathbf{s}} \in \mathbb{Z}_q^n$. □

7 Implémentation sur Gurobi

Le solveur de problèmes d'optimisation que nous allons utiliser pour résoudre numériquement des instances de LWE transformées en programmation en nombres entiers est Gurobi. Gurobi s'intègre directement comme bibliothèque dans Python : il est donc assez facile d'approche. Notons que que l'entièreté des fichiers et des scripts auxquels nous ferons référence est disponible dans le dépôt GitHub du projet, accessible en [14].

7.1 Initialisation

Le fichier le plus important, soit le fichier d'implémentation *solve_lwe*, est notre fichier de résolution de base basé sur l'article de Shirase. Nous l'avons donc glissé en annexe A. Il prend en entrée une instance $(A, \mathbf{b})$ de LWE, ainsi que son paramètre q , régissant l'espace $\mathbb{Z}_q$ sur lequel on travaille. Les paramètres t et *time_limit* sont des paramètres plus «ajustables» : t régit la borne sur notre erreur, tandis que *time_limit* mets une limite de temps au modèle pour qu'il ne s'éternise

pas sur une instance excessivement longue à résoudre. S’il arrive à trouver la solution à l’instance correspondant aux paramètres d’entrées, il retournera le vecteur de solution $\hat{s}$, le vecteur d’erreur e , ainsi que le nombre de «nodes» de Branch-and-Bound explorées par Gurobi et le temps de résolution du problème. Autrement, le solveur retourne des valeurs vides, ainsi que le nombre de nodes et le temps de résolution du problème.

Tel que mentionné plus tôt, un système LWE bien conditionné peut nous garantir, avec une très haute probabilité, qu’il existe une seule solution. Comme cette probabilité est calculable via

$$(q^n - 1) \left(\frac{4t + 1}{q} \right)^m$$

où m, n, q, t sont les paramètres de notre système LWE, il est possible d’ajuster notre système pour qu’il n’ait qu’une seule solution, et ce à la probabilité de notre choix [15].

Les systèmes d’encryption modernes garantissent cette unicité à une probabilité de 2^{-128} [2]. Cela est inutilement robuste pour notre solveur et pourrait causer des erreurs numériques : choisissons donc une probabilité de 2^{-12} qu’il y ait deux solutions, soit d’approximativement un sur 4000 ($\frac{1}{4096}$). Comme nous comptons effectuer environ une centaine de tests, la probabilité de collision est négligeable. On peut donc poser

$$(q^n - 1) \left(\frac{4t + 1}{q} \right)^m \leq 2^{-12} \quad (18)$$

et selon le type d’instance que l’on désire¹¹, il sera possible de générer une matrice qui sera forcément valide. Tout cela nous permet de limiter la recherche de solutions de Gurobi à une seule : dès qu’il en trouve une, il est donc possible de terminer l’exécution du programme, nous économisant du temps de calcul. C’est pour cela que dans le code, nous limitons le nombre de solutions via l’attribut *SolutionLimit*.

Remarque 16. Notons qu’il est possible de qualifier ce calcul d’en «pire cas». En effet, le paramètre t génère une loi uniforme dans [15] (ce qui ressemble plus souvent à ce qui est adopté en pratique), alors que nous travaillons plutôt avec une gaussienne ici, question de s’aligner le plus possible avec la théorie. Pour pallier à ce problème, nous générerons donc une distribution gaussienne entière tronquée depuis le paramètre t qui sera proposé en entrée. Via cette approche hybride, nous nous assurons tant de l’unicité de la solution que de la modélisation la plus fidèle du problème théorique. Nous fixerons donc une $\mathcal{N}(0, \sigma)$ tronquée en t avec $t = 3\sigma \implies \sigma = t/3$.

Sur un tout autre sujet, il était impératif de vérifier la validité du solveur avant de faire des tests plus poussés sur ce dernier. Un tout petit exemple numérique, tiré de [26], a donc été employé.

11. Disons que l’on désire une matrice A avec beaucoup de colonnes, il est possible de poser un très grand n et d’ajuster les autres paramètres en conséquence.

Comme la solution était déjà disponible, il était facile de comparer la sortie de notre solveur avec ce qui était proposé dans la littérature. Cet exemple peut être aperçu dans le fichier *test_manuel*, et il a été passé avec brio.

7.2 Créations d’instances, validation et script final.

Comme nous désirons analyser la performance de Gurobi sur LWE, il est nécessaire de cartographier son efficacité sur des problèmes aux paramètres différents : cela nous permettra donc de déterminer sur quels types d’instances il est efficace et à quel moment il se met à ralentir. Cette section se concentrera donc sur la création d’instances aux paramètres variés.

Pour commencer, remarquons que le code pour cette section sera séparé en deux catégories de modules : la première ne sera qu’un fichier unique nommé *gen_instance*, tandis que la seconde consistera en une collection de générateurs de paramètres valides. Le générateur d’instance n’est pas compliqué : il ne fait que créer une instance d’un problème LWE avec les paramètres qu’il reçoit en entrée. Les générateurs de paramètres, quant à eux, s’assurent de créer des quadruplets de paramètres valides qu’ils enverront ensuite au générateur d’instance. L’utilisateur peut donc générer automatiquement des combinaisons de paramètres valides car ces derniers implémentent la formule présentée en 18. Nous respectons donc le 2^{-12} fixé, mais cela pourrait être ajusté si l’on veut plus tard faire des simulations à plus grande échelle. La section des générateurs de paramètres est particulièrement modulaire : il est possible d’en ajouter à l’infini selon le type de problème que l’on veut analyser.

De plus, nous avons aussi ajouté un module qui ne fait que tester directement si Gurobi a bel et bien trouvé la bonne solution à un problème : cela nous permet d’avoir une garantie sur les résultats que nous désirerons analyser plus tard. Le fichier contenant cette vérification est appelé *test_auto*.

Compte tenu de tous ces petits modules, nous pouvons maintenant créer notre script de base, soit la fonction *test_style_instance* au début du fichier *benchmark*. Elle prend en entrée le type de test que l’on veut effectuer, autant sa fonction que le nom de son test, prend la configuration de paramètres du test, et prend le nombre de fois que l’on veut effectuer un test avec. Nous avons pour l’instant fixé cette valeur à 3. Ensuite, pour un test fixé, elle génère des paramètres selon le test que l’on veut effectuer en appelant sa fonction de génération *gen_parametres* appropriée, qui va ensuite créer une instance via *gen_instance*. Gurobi va résoudre ce problème, il sera vérifié via *test_auto*, et les informations pertinentes au test seront retournées de manière à être facilement transférable dans un fichier CSV. Dans le fichier *benchmark*, il est aussi possible de trouver le script principal qui produira le fichier CSV que nous utiliserons pour produire les graphiques que nous présenterons dans la prochaine section.

7.3 Tests et résultats préliminaires

Pour cette section de tests, nous travaillons sur une machine Windows utilisant Gurobi 13.0.2. Nous utilisons les 12 threads d'un Ryzen 5 8645HS d'AMD avec 16gb de RAM.

En premier lieu, une série de tests bien conditionnés ont été effectuées. Dans ces tests, le paramètre n était fixé à 10, le paramètre q à 101, tandis que le paramètre m était graduellement incrémenté. Pour ce qui est de t , il respectait l'équation 18, et donc était aussi incrémenté au fur et à mesure qu'augmentait m . On peut donc le voir comme un test dynamique : deux paramètres variaient simultanément. Nous pouvons introduire cette variation sur les graphiques ci-dessous ¹² :

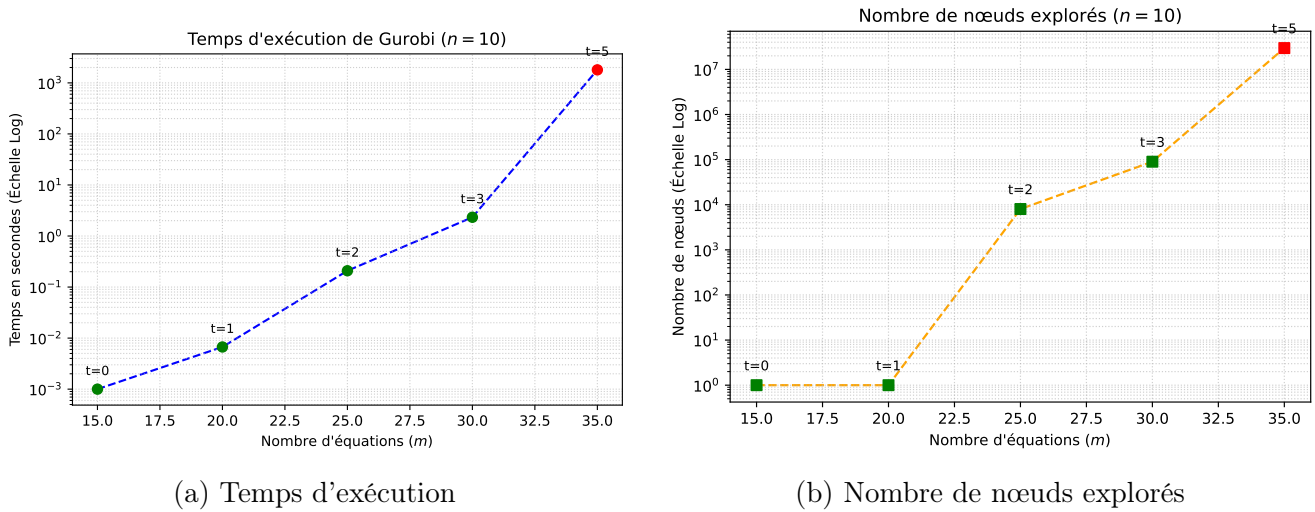


FIGURE 4 – Analyse de la performance de Gurobi pour m, t croissant et $n = 10$.

Directement, il nous est possible de confirmer que la complexité de résolution de notre modèle LWE est exponentielle selon les paramètres t et m . En produisant des graphiques à échelle logarithmique, nous créons des relations avec un semblant de linéarité. Par défaut, nous avons limité le temps d'exécution de Gurobi à 30 minutes, soit 1800 secondes : nous pouvons apercevoir que le couple $m = 35, t = 5$ n'a pas terminé son exécution dans les temps impartis. Somme toute, ces graphiques nous permettent de comprendre que notre modèle a pour l'instant ses limites : il ne peut résoudre de grandes instances du problème LWE. Sur les prochaines paires de graphiques, nous analysons l'impact de la variation d'un paramètre individuel à la fois : pour se faire, il était cependant impossible de toujours garantir un bon conditionnement des instances.

En figure 5, un système aux paramètres n, q et t fixe a été créé, et seule la valeur de m était variable. On remarque ici que m n'a pas particulièrement d'influence sur Gurobi : malgré les augmentations successives de sa valeur, le temps d'exécution ainsi que le nombre de nœuds explorés n'augmentait pas vraiment : il est donc difficile d'en tirer une conclusion. Cependant, en

12. À titre de convention, nos cercles sont verts si Gurobi a trouvé la bonne solution, tandis qu'ils sont rouges s'il n'a pas trouvé la solution, donc si la solution qu'il a trouvée est mauvaise ou qu'il n'en a pas trouvée une.

faisait varier ce paramètre sur une plus petite échelle, il nous arrivait d'obtenir de basses valeurs de m pour lesquelles le système ne pouvait être résolu. À force de raffiner notre recherche sur cette valeur critique, nous sommes arrivés au résultat présenté en figure 6.

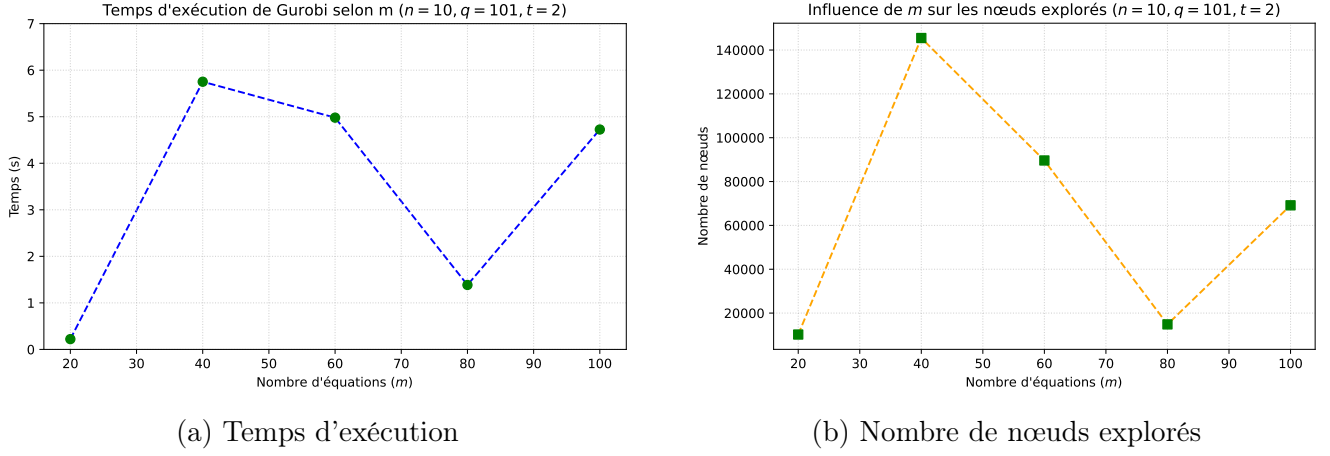


FIGURE 5 – Influence du paramètre m sur la performance de Gurobi (grande échelle).

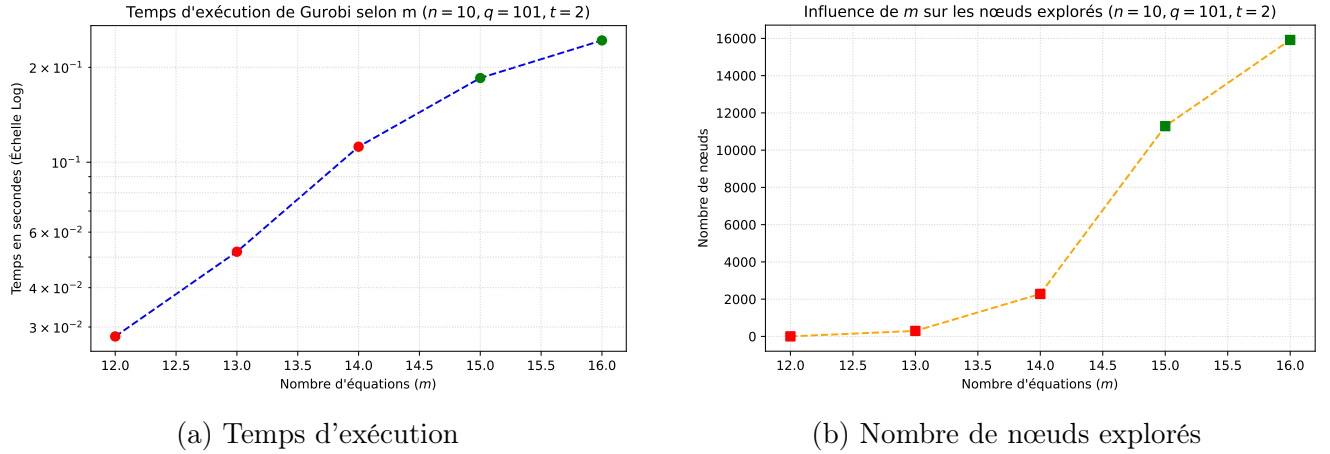


FIGURE 6 – Influence du paramètre m sur la performance de Gurobi (petite échelle).

Nous pouvons remarquer qu'à plus petite échelle, lorsque les valeurs de m sont particulièrement basses et proches de $n = 10$, le système ne retourne pas la solution valide. Cela confirme l'hypothèse que nous avons placée plus haut : lorsque l'on ne s'assure pas que le système est bien conditionné, il est possible qu'il existe plus d'une solution. Par exemple, lorsque $m = 13, n = 10$, le nombre d'équations m n'est pas assez grand pour restreindre le nombre d'inconnues n du vecteur de solution. Comme nous avons codé notre solveur Gurobi pour toujours ne retourner que la première solution, il y a une grande chance que la première solution trouvée ne soit pas la bonne. C'est exactement cela qui se passe ici : il retourne une solution excessivement rapidement, mais cette dernière n'est pas valide.

Naturellement, il suivait de créer un système similaire où l'on ferait varier le paramètre n plutôt

que m . Afin d'observer son impact de manière efficace, il était important de fixer un m n'étant pas trop élevé, autrement nous entrerions dans des cas plus lents tels que présentés en figure 4. Nous avons donc fixé $m = 20, q = 149$ et $t = 2$.

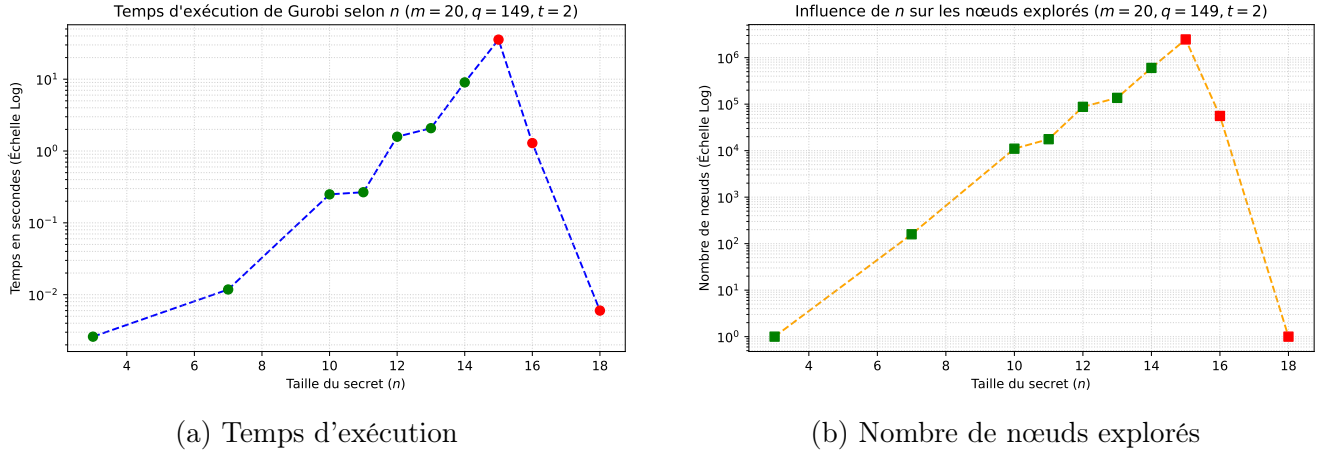


FIGURE 7 – Influence du paramètre n sur la performance de Gurobi pour $m = 20$ et $t = 2$.

Remarquons que nos résultats sont encore une fois fidèles à la théorie : le temps de calcul augmente exponentiellement lorsque la taille du secret n augmente linéairement : c'est de là que provient la difficulté du problème LWE. Cependant, remarquons aussi quelque chose d'autre d'intéressant : lorsque n arrive dans la zone critique près de la valeur de m , Gurobi ne retourne pas la bonne solution, de manière identique à ce que nous avons vu précédemment. Cependant, le calcul de cette mauvaise solution est de plus en plus rapide : en effet, plus n se rapproche de m , plus le nombre de mauvaises solutions est grand, et donc plus il est rapide de tomber sur une solution quelconque. En $n = 18$, on trouve donc cette mauvaise solution beaucoup plus efficacement qu'en $m = 15$.

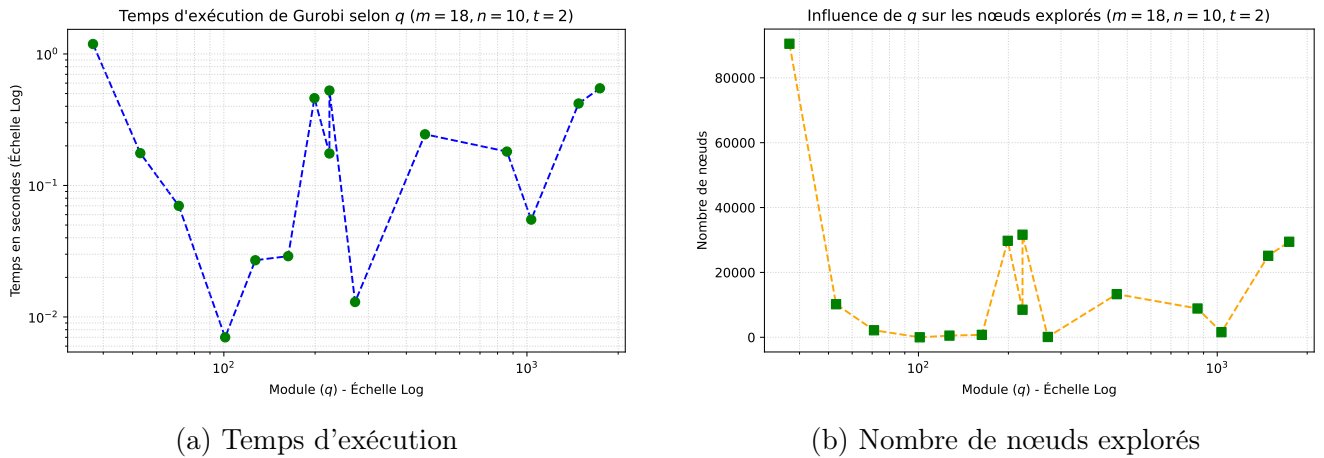


FIGURE 8 – Influence du paramètre q sur la performance de Gurobi.

En continuant dans la même veine, étudions le paramètre q . Ce dernier est beaucoup moins

intéressant : en figure 8, nous remarquons que le temps de calcul ainsi que les noeuds ne semblent pas suivre de relation particulière quant à l'accroissement de la valeur de q . On peut donc attribuer les variations dans l'efficacité du calcul au hasard introduit par la générations d'instances aléatoires.

Pour terminer, il ne restait plus qu'à analyser l'impact du paramètre t sur le solveur. Nous travaillons ici avec un $q = 127$ et un $n = 11$. Initialement, il est facile de remarquer que lorsque $m = 15$ et $t \geq 2$, le problème est mal conditionné. C'est ce qui fait que Gurobi ne peut résoudre ces instances. Autrement, il est possible de remarquer une augmentation générale du temps de calcul lors t augmente. Cela est complètement attendu : plus t augmente, plus la borne d'erreur augmente, et donc plus Gurobi doit gérer de possibilités.

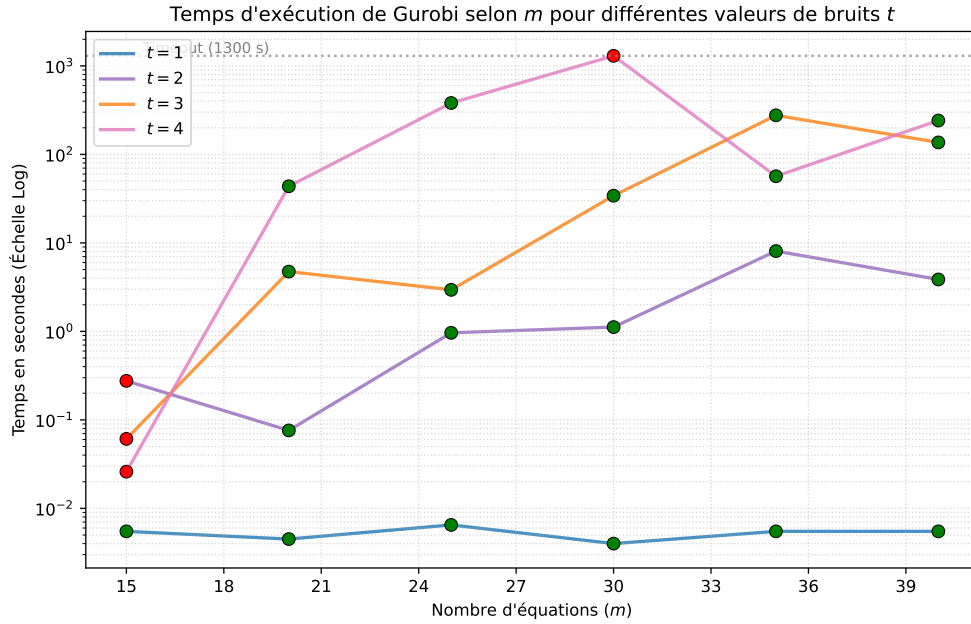


FIGURE 9 – Évolution du temps de calcul en fonction de m , selon un t variable.

7.4 Performance face aux cryptosystèmes actuels

Il est difficile de quantifier directement la performance de notre modèle de PNE sur des instances de systèmes d'encryption d'actualités utilisant le problème LWE pour deux raisons principales :

1. C'est une tâche excessivement difficile algorithmiquement. Rappelons que Kyber, soit l'implémentation pratique de LWE la plus étudiée, a été désigné comme étant le prochain standard de protocole d'échange de clé par le NIST. Cette certification nous assure une chose : tenter de déterminer une solution en un temps raisonnable à un problème conditionné avec les mêmes paramètres que Kyber est une tâche que l'on croit pour l'instant impossible. Par de simples contraintes de temps, il n'est pas envisageable de tester notre modèle face à Kyber.

2. Kyber n'utilise pas la version classique de *learning with errors* ! Tel que présenté en section 4.5, c'est le problème Module-LWE qui est implémenté. La réduction de Shirase ne s'appliquant qu'au problème de base, il n'est pas possible d'établir une relation directe entre Kyber et notre modèle de PNE. Cependant, il est possible de les rapprocher : commençons donc par présenter Module-LWE.

Définition 37. (Module-LWE) Le problème *Module-LWE*, que l'on abrège à $\text{MLWE}(k, l, n, q, B)$, se pose comme suit. Soit $A \in_R R_q^{k \times l}$, $\mathbf{s} \in_R R_q^l$ et $\mathbf{e} \in_R S_B^k$, avec $k > l$, $B \ll q/2$ et $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$. En ne connaissant que A et $\mathbf{b} = A\mathbf{s} + \mathbf{e} \in R_q^k$, le but du problème est de trouver $\mathbf{s}$.

Pour ne pas débiter de discussion abstraite sur la théorie des anneaux, contentons-nous de dire que $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$ est l'ensemble de tous les polynômes de degré inférieur à n dont les coefficients sont dans $\mathbb{Z}_q$. De plus, voyons simplement S_B comme l'ensemble des polynômes dont les coefficients sont bornés par B et dont le degré est aussi strictement plus petit que n .

Depuis cette définition, il est possible de remarquer que la matrice A dans MLWE est une matrice constituée de polynômes. Heureusement, il est possible de la transformer en matrice composée uniquement d'entiers compris dans $\mathbb{Z}_q$ pour qu'elle devienne interprétable par Gurobi [15]. Définissons d'abord une catégorie particulière de matrices.

Définition 38. Soit un vecteur $\mathbf{a} = (a_0, a_1, \dots, a_{n-1})$. Une matrice anticirculante de taille $n \times n$ engendrée par le vecteur $\mathbf{a}$ est une matrice carrée formellement définie par :

$$\mathbf{A} = \begin{bmatrix} a_0 & -a_{n-1} & -a_{n-2} & \cdots & -a_1 \\ a_1 & a_0 & -a_{n-1} & \cdots & -a_2 \\ a_2 & a_1 & a_0 & \cdots & -a_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \cdots & a_0 \end{bmatrix}$$

On notera cette matrice par $\overline{\text{circ}}(\mathbf{a})$.

Continuons ensuite avec deux propositions.

Proposition 8. Soit $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$. Un polynôme $f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} \in R_q$ peut être représenté par son vecteur de coefficients de longueur n : $\mathbf{f} = (a_0, a_1, a_2, \dots, a_{n-1})$.

Proposition 9. Soit $a(x)$ et $b(x)$ deux polynômes dans R_q . Il est possible d'exprimer la multiplication polynomiale $c(x) = a(x)b(x)$ dans R_q via $\mathbf{c} = \overline{\text{circ}}(\mathbf{a})\mathbf{b}$, où $\mathbf{b}$ et $\mathbf{c}$ sont les vecteurs représentatifs des polynômes $c(x)$ et $b(x)$ et où $\overline{\text{circ}}(\mathbf{a})$ est la matrice anticirculante du vecteur $\mathbf{a}$ représentatif de $a(x)$ ¹³.

13. La preuve est disponible dans [15].

Compte tenu de ces énoncés, analysons le système d'équations $A\mathbf{s} + \mathbf{e} = \mathbf{b}$: créons un problème MLWE avec les paramètres $k = 3, l = 2, n = 4, B = 1$ et $q = 37$. Visuellement, nous obtenons :

$$\begin{bmatrix} a_{1,1}(x) & a_{1,2}(x) \\ a_{2,1}(x) & a_{2,2}(x) \\ a_{3,1}(x) & a_{3,2}(x) \end{bmatrix} \begin{bmatrix} s_1(x) \\ s_2(x) \end{bmatrix} + \begin{bmatrix} e_1(x) \\ e_2(x) \\ e_3(x) \end{bmatrix} = \begin{bmatrix} b_1(x) \\ b_2(x) \\ b_3(x) \end{bmatrix}.$$

Isolons ensuite la première ligne du système :

$$a_{1,1}(x) \cdot s_1(x) + a_{1,2}(x) \cdot s_2(x) + e_1(x) = b_1(x),$$

et remarquons qu'il est possible de transformer cette formulation polynomiale. Par les propositions que nous avons posé plus tôt, nous pouvons obtenir la représentation vectorielle suivante :

$$\begin{bmatrix} \overline{\text{circ}}(a_{1,1}) & \overline{\text{circ}}(a_{1,2}) \end{bmatrix} \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{bmatrix} + \mathbf{e}_1 = \mathbf{b}_1.$$

En appliquant ce même raisonnement sur toute la matrice, nous pourrions alors obtenir :

$$\begin{bmatrix} \overline{\text{circ}}(a_{1,1}) & \overline{\text{circ}}(a_{1,2}) \\ \overline{\text{circ}}(a_{2,1}) & \overline{\text{circ}}(a_{2,2}) \\ \overline{\text{circ}}(a_{3,1}) & \overline{\text{circ}}(a_{3,2}) \end{bmatrix} \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{bmatrix} + \begin{bmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \\ \mathbf{e}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}.$$

Ce système correspond exactement à un problème LWE classique. À des fins d'analyse, posons

$$\hat{A} = \begin{bmatrix} \overline{\text{circ}}(a_{1,1}) & \overline{\text{circ}}(a_{1,2}) \\ \overline{\text{circ}}(a_{2,1}) & \overline{\text{circ}}(a_{2,2}) \\ \overline{\text{circ}}(a_{3,1}) & \overline{\text{circ}}(a_{3,2}) \end{bmatrix}, \quad \hat{\mathbf{s}} = \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{bmatrix} \quad \text{et} \quad \hat{\mathbf{e}} = \begin{bmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \\ \mathbf{e}_3 \end{bmatrix}.$$

Extrayons les dimensions de ce système. Remarquons pour commencer que les paramètres B et q n'ont aucune influence sur cette transformation. Ensuite, remarquons que $\hat{\mathbf{s}}$ est de dimension $2 \cdot 4 = 8 = l \cdot n$ et que $\hat{\mathbf{e}}$ est de dimension $3 \cdot 4 = 12 = k \cdot n$. En posant $\hat{m}, \hat{n}, \hat{q}$ et $\hat{B}$ les paramètres du système LWE classique correspondant à un problème MLWE, nous pouvons affirmer que $\hat{m} = k \cdot n$, que $\hat{n} = l \cdot n$, que $\hat{q} = q$ et que $\hat{B} = B$.

Le NIST quantifie la sécurité d'un cryptosystème selon une échelle débutant à 1 et terminant à 5. Un système au niveau 1 est d'une bien plus basse sécurité qu'un système de niveau 5, mais il requiert beaucoup moins de puissance de calcul à implémenter [6]. Tous les systèmes ont donc leurs usages : il s'agit simplement de choisir le système approprié à une tâche quelconque.

Remarque 17. Un cryptosystème de niveau 1 reste excessivement sécuritaire : pour qu'un procédé cryptographique soit dans cette catégorie, il doit être au minimum aussi difficile que le crypto-

système AES-128. À titre d’image, notons qu’à son pic historique, le réseau bitcoin, soit le plus grand réseau de calcul jamais construit par l’humanité, roulait à 10^{21} hashages par seconde, soit environ 2^{70} opérations par secondes. Une seule clé AES-128 admet, comme son nom l’indique, 2^{128} possibilités. Pour tester toutes ces possibilités, il faudrait alors $2^{128} \div 2^{70} = 2^{58}$ secondes de calcul : cela équivaut à environ 9 milliards d’années (l’univers en a environ 13!) [9].

Le cryptosystème Kyber est justement flexible : il admet trois niveaux de sécurité via ses trois variations, soit ML-KEM-512, ML-KEM-768 et ML-KEM-1024. Dans le tableau ci-dessous, nous présentons les paramètres utilisés par ces dernières.

Algorithme	Niveau NIST	n	q	k	B
ML-KEM-512	1	256	3329	2	3
ML-KEM-768	3	256	3329	3	2
ML-KEM-1024	5	256	3329	4	2

TABLEAU 1 – Paramètres de ML-KEM selon son niveau de sécurité.

Remarque 18. Kyber a été conçu spécifiquement pour ne travailler qu’avec des matrices carrées, où $k = l$. Cela varie légèrement de MLWE, où nous avons $k > l$. L’unicité de la solution reste garantie comme nous travaillons avec de très grandes matrices et un très grand q , sous une très petite erreur.

À travers les trois algorithmes, le degré des polynômes $n = 256$ et le module $q = 3329$ restent constants. Remarquons aussi que la borne d’erreur est toute faible : elle ne dépasse jamais 3. Kyber travaille avec une distribution binomiale centrée tronquée plutôt qu’une gaussienne. Cela permet d’assurer que les valeurs ne peuvent pas théoriquement aller plus loin que l’intervalle $[-B, B]$ ¹⁴.

Observons que le principal changement entre les trois algorithmes est un simple ajustement du paramètre k , déterminant la taille de la matrice polynomiale. Cette valeur passe de 2, à 3, puis à 4. C’est le principal moteur de difficulté. Remarquons aussi que l’erreur diminue entre les niveaux 1 et 3. Ce n’est qu’une mesure de compensation : comme la taille de la matrice a assez augmenté, il est possible de diminuer la borne d’erreur tout en gardant une certification de sécurité de niveau 3 ¹⁵.

En modifiant ces trois implémentations de Kyber en problèmes LWE classiques via la transformation présentée plus haut, il est possible d’obtenir les paramètres suivants :

14. Cette distribution sera implémentée dans le code des prochaines sections, mais remarquons que cela était très similaire à ce que nous avons déjà fait, en contraignant la Gaussienne à un certain écart-type.

15. Ces niveaux sont échellonnés par le NIST selon des critères précis. On pourrait donc supposer que qu’un $k = 3$ et un $B = 3$ engendrait une difficulté de niveau 4, mais qu’il était plus intéressant d’échelonner le tout sur des difficultés de 1, 3, 5. Le paramètre B a donc été ajusté en conséquence.

Algorithme	$\hat{m} = k \cdot n$	$\hat{n} = l \cdot n$	$\hat{q} = q$	$\hat{B} = B$
ML-KEM-512	512	512	3329	3
ML-KEM-768	768	768	3329	2
ML-KEM-1024	1024	1024	3329	2

TABLEAU 2 – Dimensions du système LWE classique équivalent aux instances de ML-KEM.

Si l'on voulait simuler une attaque sur une instance de Kyber, il faudrait alors tenter de briser une instance de LWE classique avec des paramètres m et n étant de l'ordre de plus de 500 ! Pour l'instant, notre solveur semble prendre énormément de temps lorsque les valeurs de m et de n dépassent la trentaine : comme la difficulté du problème LWE est supposée exponentielle, il est naturel de croire qu'une instance de l'ordre de Kyber ne sera pas calculable efficacement par notre modèle actuel. Notons par contre que les problèmes MLWE, lorsque transformés en problème LWE, admettent une certaine régularité. Il serait donc intéressant d'ajuster notre solveur en conséquence. Dans la prochaine section, nous proposerons donc des améliorations à notre modèle de PNE, tout en faisant progressivement grandir la taille des instances sur lesquelles nous travaillons.

8 Améliorations

Nous utilisons encore une fois Gurobi 13.0.2, sous les 6 des 12 threads d'un Ryzen 5 8645HS d'AMD¹⁶. Cependant, pour cette section-ci, nous travaillons sur un système WSL, soit un système Linux exécuté directement à l'intérieur de Windows. Ce dernier est initialisé pour utiliser toute la capacité possible de notre processeur, mais seulement 12 des 16 gb de RAM disponibles. La configuration système est donc légèrement réduite par rapport à ce qui a été effectué lors des tests initiaux.

8.1 Alignement aux standards industriels

À travers cette section, nous tentons le plus possible de nous rapprocher des instances générées par Kyber. Pour commencer, nous changeons la distribution gaussienne tronquée que nous avons initialisée pour l'erreur pour une distribution binomiale centrée bornée (DBC) que l'on nommera aussi B_n , où n est le paramètre de dispersion. Il est possible de définir B_n comme suit [2] :

$$\text{Échantillonner } (a_1, \dots, a_n, b_1, \dots, b_n) \leftarrow \{0, 1\}^{2n}$$

$$\text{et retourner } \sum_{i=1}^n (a_i - b_i).$$

16. Ces derniers sont manuellement limités sous Gurobi : cela semblait offrir plus de performance.

Cela permet de contenir les coefficients du vecteur d'erreur $\mathbf{e}$ entre $[-n, n]$, car chaque coefficient est échantillonné selon le procédé que l'on vient de poser plus haut. Autrement, le paramètre q sera fixé à 3329 lors de tous nos tests pour rester fidèle aux standards de l'industrie présentés plus haut.

Une légère mais non-négligeable optimisation qui sera appliquée sur notre modèle de base est celle de l'implémentation du modulo symétrique [15], que nous définissons comme suit :

Définition 39. Soit $q \geq 3$ un entier impair, et soit $r \in \mathbb{Z}_q$. On définit le *modulo symétrique* de r par :

$$r \bmod q = \begin{cases} r & \text{si } r \leq (q-1)/2 \\ r - q & \text{si } r > (q-1)/2 \end{cases}$$

L'opération $\bmod q$ s'étend naturellement à tous les entiers : si $z \in \mathbb{Z}$, alors :

$$z \bmod q = (z \bmod q) \bmod q.$$

Les articles purement théoriques n'en font jamais allusion, mais les systèmes pratiques implémentent toujours leur modulo de cette manière - Kyber n'en fait pas exception. Remarquons que $-(q-1)/2 \leq r \bmod q \leq (q-1)/2$. Par exemple, si $q = 13$ et $r \in \mathbb{Z}_{13}$, alors nous avons $-6 \leq r \bmod 13 \leq 6$. Avec un mod classique, nous aurions $0 \leq r \bmod q \leq 12$. Le nombre d'entiers entre ces bornes reste forcément le même peu importe l'opérateur modulo choisi, mais les valeurs sont beaucoup moins élevées avec $\bmod q$ qu'avec un mod classique.

Cette façon de concevoir les modulus est beaucoup plus en lien avec les distributions d'erreurs que nous avons présentées depuis le début : elles sont toutes centrées autour de 0. Par exemple, garder le modulo classique avec une binômiale centrée engendrerait des erreurs conceptuelles. Un coefficient secret quelconque pourrait être très loin de 0, soit à 3328, mais en subissant une erreur de +1, il se retrouverait directement sur 0. Avec un modulo centré, nous passerions de 1664 à -1664 : la représentation de la distance d'un point à l'origine est donc beaucoup plus fidèle à la réalité en modulo centré. Malgré tout, tel que nous l'avons présenté plus tôt, le modèle de Shirase à modulo classique ne causait aucun problème d'exactitude, le modèle ne cherchant pas à minimiser le vecteur secret $\mathbf{s}$ mais bien le vecteur d'erreur $\mathbf{e}$. Ce changement ne s'agit donc pas d'une reformulation nécessaire au bon fonctionnement du solveur, mais est vraiment uniquement d'ordre conceptuel.

À titre de comparaison et de validation, implémentons ce centrage modulaire et comparons le à notre implémentation sous modulo classique. Nous travaillons avec un $q = 3329$ et une distribution binomiale centrée avec $t = 2$, et un $n = 11$. Le graphique que nous présentons ci-dessous présente la performance du solveur *solve_lwe* classique versus celle du solveur *solve_lwe_cmod* implémentant

le centrage modulaire. 5 valeurs de m ont été utilisées, sur lesquelles 30 tests par valeurs de m (correspondant aux seeds 1 à 30) ont été effectués. Ces 30 tests nous garantissent que notre échantillon est significatif tout en ne prenant pas trop de temps à être exécuté.

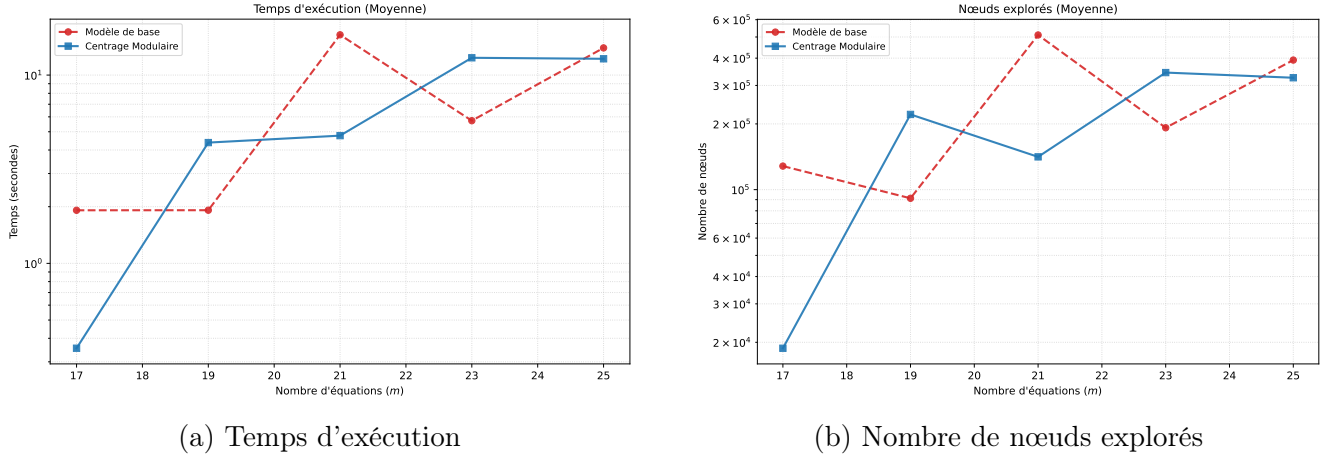


FIGURE 10 – Comparaison (en moyenne) du modèle de base avec le modèle CMOD.

Il semble difficile de tirer une conclusion lorsque nous regardons uniquement les statistiques sur les tests calculés avec la moyenne. Il suffit d'y avoir un test qui prenne énormément de temps pour faire exploser la valeur de cette dernière, tant pour les noeuds que pour le temps. Nous avons donc décidé d'extraire aussi les résultats médians.

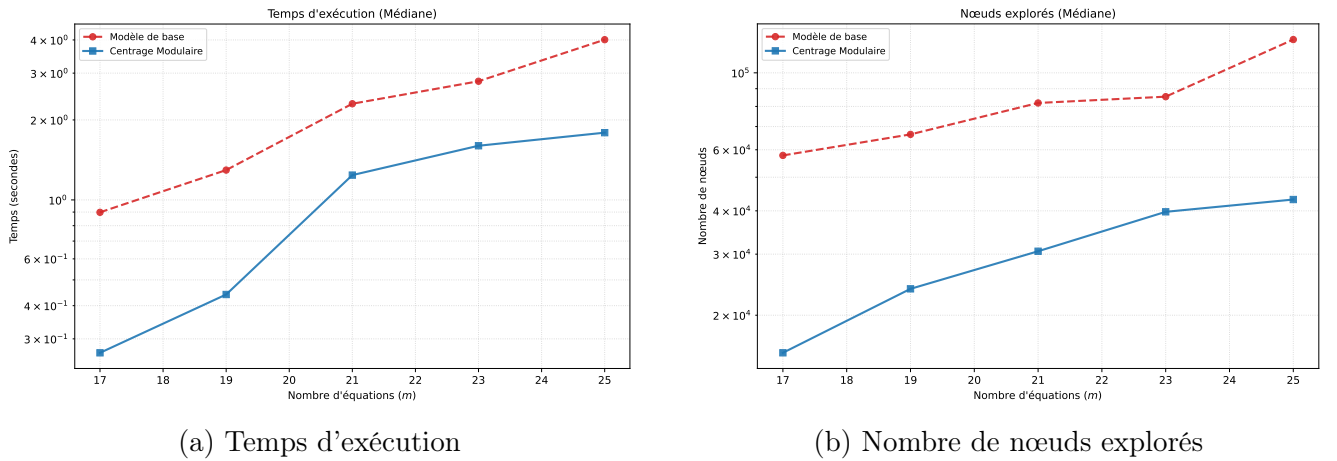


FIGURE 11 – Comparaison (médiane) du modèle de base avec le modèle CMOD.

Nous observons alors une amélioration significative suite à l'implémentation du centrage modulaire pour notre solveur : les instances résolues avec le solveur *solve_lwe_cmod* ont leur solution découverte deux à trois fois plus rapidement. Bien qu'il soit difficile de déterminer exactement comment cette implémentation rend Gurobi plus efficace, nous pouvons supposer que cela découle de la variable de compensation f_i . Les i contraintes principales du modèle de Shirase sont chacune conçues via une somme $\sum_j w_{i,j}$ qui est balancée (on peut le voir comme une remise à niveau du

modulo) par un terme qf_i . Si ces sommes sont grandes, alors les valeurs de f_i devront être grandes pour compenser, et Gurobi explorera probablement plus de branches pour les f_i . Si ces sommes sont plus petite, ce qu'elles deviennent suite au centrage des coefficients $w_{i,j}$ autour de 0, il y a plus de chance que les valeurs des f_i soient plus petites, et donc que Gurobi les trouvent plus rapidement.

Remarque 19. Depuis le début de ce rapport, il est possible d'observer une relation de proportionnalité entre les noeuds et le temps d'exécution. Nous commencerons donc à présenter le graphique d'un seul des deux lors de nos analyses, selon ce qui concorde le mieux avec les propos véhiculés.

Pour le reste de cette étude, nous travaillerons uniquement avec mods : notre standard deviendra donc solve_lwe_cmod. Nous écrirons tout de même mod pour dire mods par souci de simplicité. De plus, seule la DBC sera utilisée, et le paramètre q sera fixé à 3329, respectant Kyber. Les paramètres variables resteront donc m, n et t , la variable t ne pouvant prendre que les valeurs 2 et 3.

8.2 Réduction directe de la matrice W

En section 3.4, nous avons présenté l'algorithme LLL. Son temps d'exécution polynomial et sa polyvalence nous firent le choisir en premier lorsqu'il fallut commencer à améliorer le modèle de PLNE que nous avons posé plus tôt.

Pour commencer, une fonction nommée `solve_lwe_lll` a été implémentée. À travers cette dernière, nous utilisons la librairie python `fypLLL` afin de pouvoir exécuter l'algorithme LLL sans avoir à le coder manuellement. Dans le modèle en PLNE de LWE de base, nos contraintes principales prennent la forme suivante :

$$\sum_{j=1}^n w_{i,j}x_j - x_{n+i} + qf_i = -u_i.$$

Rappelons que le côté droit de l'équation, soit la valeur $-u_i$, est une valeur contenue dans $\mathbb{Z}_q$. Dans notre cas, comme nous travaillons en modulo centré avec $q = 3329$, cette valeur est contenue dans l'intervalle discret $\{-1664, -1663, \dots, 0, \dots, 1663, 1664\}$. Il est donc naturel de vouloir obtenir une somme $\sum_{j=1}^n w_{i,j}x_j$ assez basse : si cette dernière est petite, alors le terme de compensation qf_i se devra d'être lui aussi petit. Comme q est une constante, f_i sera aussi une plus petite valeur. Gurobi prendra alors moins de temps à déterminer la valeur de f_i , comme il oriente naturellement sa recherche vers des valeurs proches de zéro. En passant la matrice W directement à travers cet algorithme, nous supposons que via la condition de taille, ses coefficients deviendraient plus petits.

Des tests pratiques ont été effectués avec $n = 11, t = 2$ et avec un m variable. Pour tout

$m < 22$, Gurobi retournait une erreur : les instances étaient insatisfaisables, car les coefficients de la matrice de passage U dépassaient les valeurs numériques maximales prises en charge par Gurobi. En creusant un peu, il était possible de réaliser que comme $W \equiv A_1 A_0^{-1}$, alors W était de taille $(m-n) \times n$. À $m < 2n$, nous obtenions alors une matrice large, pour laquelle il y a plus de colonnes que de lignes. Or, il est impossible d'appliquer LLL sur ce genre d'instance [?], expliquant alors les coefficients dégénérés de U .

À $m \geq 22$, nous obtenons cette fois-ci des solutions. À titre de comparaison, présentons dans le tableau ci-dessous les performances du solveur CWE à centrage modulaire versus le solveur CWE avec LLL sur W . Notons que le solveur LLL utilise aussi le centrage modulaire : il n'est donc pas désavantagé.

TABLEAU 3 – Comparaison du modèle CMOD avec le modèle LLL ($n = 11, q = 3329, t = 2$)

Modèle	m	Temps moyen (s)	Temps médian (s)	Statut
CMOD	18	4.3951	0.9208	Résolu
	22	1.6176	1.7051	Résolu
	24	5.8766	1.6375	Résolu
LLL	18	0.3249	0.3249	Non résolu
	22	13.1743	6.3400	Résolu
	24	15.2254	13.7519	Résolu

D'un seul coup d'oeil, il est clair que le solveur LLL est moins efficace que le solveur CMOD, peu importe que le système soit déterminé, sous-déterminé ou surdéterminé. Cet échec s'explique par une utilisation erronée de l'algorithme LLL. Rappelons que ce dernier est un algorithme de réduction de base sur un réseau. Cependant, la matrice W n'est pas représentative du réseau du problème CWE ! Cette dernière n'encode aucunement la structure modulaire du problème. Dans la prochaine section, nous proposons une alternative pour contourner ce problème en retrouver une base plus fidèle au problème.

8.3 Contruction algébrique alternative

8.3.1 Modélisation mathématique

Les attaques primales sur le cryptosystème CWE sont basées sur la réduction de CWE à un problème de BDD. Habituellement, BDD est ensuite réduit à Unique-SVP via la technique d'encastrement de Kannan. Cependant, nous démontrerons ici qu'il est possible de mener une attaque primale sans la dernière réduction, le problème de BDD étant directement solvable par Gurobi.

L'article ressemblant le plus à notre travail actuel est cité en [8] : il date du 22 juin 2026 (!)

et a été écrit par Shuxian Jiang. Dans celui-ci, le problème LWE est transformé en un problème d'optimisation d'une manière similaire à celle de Shirase. Le partitionnement A_0, A_1 de la matrice A est identique et il y a création d'une matrice $W \equiv A_1 A_0^{-1}$. C'est ensuite que leurs chemins divergent : Shirase pose un problème de PQNE pour un solveur MIP, tandis que Jiang quitte vers un modèle d'optimisation binaire quadratique sans contraintes (QUBO), qu'elle fera résoudre par une Ising Machine, soit un solveur physique s'approchant de l'ordinateur quantique.

En préparation à l'optimisation QUBO, Jiang [8] se construit une base générant le réseau représentatif d'une instance LWE quelconque. Présentons sa structure et la manière de l'obtenir.

Théorème 9. Soit A une matrice aléatoire de taille $m \times n$ initialisant un problème LWE(m, n, q, B) avec $m > n$. Soit A_0 la sous-matrice correspondant aux n premières lignes de A et soit A_1 la matrice correspondant aux $m - n$ lignes restantes. Supposons que A_0 soit inversible, et posons ensuite $W \equiv A_1 A_0^{-1}$. Il est alors possible d'obtenir une base générant le réseau représentatif du problème LWE L_A via la matrice

$$B = \begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & q \cdot I_{(m-n) \times (m-n)} \end{bmatrix}.$$

Démonstration. Soit A la matrice aléatoire d'une instance LWE et soit $\mathbf{v}$ un vecteur tel que $\mathbf{v} \in L_A$. Partitionnons ces objets de manière à ce que

$$A = \begin{bmatrix} A_0 \\ A_1 \end{bmatrix} \text{ et } \mathbf{v} = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \end{bmatrix},$$

où les parties d'indice 0 contiennent les n premières lignes de l'objet et où les parties d'indice 1 contiennent les $m - n$ dernières. Comme $\mathbf{v} \in L_A$, alors il respecte $\mathbf{v} \equiv A\mathbf{s}$ par la définition 20. Nous pouvons alors décomposer cette congruence en :

$$\mathbf{v}_0 \equiv A_0 \mathbf{s} \tag{19}$$

$$\mathbf{v}_1 \equiv A_1 \mathbf{s}, \tag{20}$$

et comme A_0 est supposée inversible, il est possible d'isoler $\mathbf{s}$ dans l'équation 19 pour obtenir :

$$\mathbf{s} \equiv A_0^{-1} \mathbf{v}_0. \tag{21}$$

En substituant l'équation 21 dans 20, nous obtenons alors, en posant $W \equiv A_1 A_0^{-1}$:

$$\mathbf{v}_1 \equiv A_1 (A_0^{-1} \mathbf{v}_0) \equiv W \mathbf{v}_0, \tag{22}$$

où il est possible de remarquer que le vecteur secret $\mathbf{s}$ a complètement été éliminé, tout comme dans

l'article de Shirase. Transformons ensuite cette congruence en une équation simple en ajoutant un vecteur d'entier arbitraire $k \in \mathbb{Z}^{m-n}$, nous obtenons alors :

$$\mathbf{v}_1 = W\mathbf{v}_0 + q\mathbf{k}. \quad (23)$$

Cette construction établit alors une condition nécessaire et suffisante pour un vecteur $\mathbf{v}$, le point lui correspondant n'appartenant à L_A que si et seulement si il est possible de le paramétrer via les variables libres $\mathbf{v}_0 \in \mathbb{Z}^n$ et $k \in \mathbb{Z}^{m-n}$ de manière à construire le système suivant :

$$\mathbf{v} = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0 \\ W\mathbf{v}_0 + q\mathbf{k} \end{bmatrix}. \quad (24)$$

Lorsque nous mettons le vecteur $[\mathbf{v}_0^\top \mathbf{k}^\top]^\top$ en évidence, nous obtenons

$$\mathbf{v} = \underbrace{\begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & qI_{(m-n) \times (m-n)} \end{bmatrix}}_B \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{k} \end{bmatrix},$$

et nous obtenons bel et bien la matrice B posée dans le théorème. Cette matrice est bel et bien une base génératrice du réseau L_A car il est possible de générer tous les points de L_A avec B . $\square$

Par cette construction algébrique, nous obtenons cette fois-ci une base B valide prête à être LLL-isée tout en éliminant encore le vecteur $\mathbf{s}$. Le prochain modèle de PQNE que nous allons construire sera donc quasiment identique à celui de Shirase, car nous cherchons encore à minimiser la norme du vecteur $\mathbf{e}$.

Remarquons cependant que le théorème 9 trouve une base valide quant à la génération d'un réseau L_A de vecteurs solutions $\mathbf{s}$, dans lequel le vecteur $\mathbf{e}$ n'existe pas. Il est donc nécessaire de modifier légèrement la construction algébrique de Jiang afin de trouver une base B' représentative du réseau d'erreurs $L_{A'}$. Nous travaillons ici avec une instance LWE où le couple $(A, \mathbf{b})$ a déjà été calculé. En posant simplement les équations

$$b_0 \equiv A_0 s + e_0 \quad (25)$$

$$b_1 \equiv A_1 s + e_1 \quad (26)$$

plutôt que les équations 19 et 20 et en repétant les ensuite les même étapes jusqu'à l'équation 22, nous obtenons alors l'équation

$$b_1 \equiv Wb_0 - We_0 + e_1 \quad (27)$$

se réduisant à

$$b_1 - Wb_0 \equiv e_1 - We_0. \quad (28)$$

Le membre de gauche de la congruence est composé uniquement des variables dont la valeur est connue, nous pouvons donc définir $u = b_1 - Wb_0 \pmod q$ et nous obtenons, en levant le modulo q comme en équation 23 :

$$e_1 = We_0 + qk + u \quad (29)$$

ce qui nous permet de trouver le système

$$e = \begin{bmatrix} e_0 \\ We_0 + qk + u \end{bmatrix} = \underbrace{\begin{bmatrix} I_n & 0 \\ W & qI \end{bmatrix}}_B \begin{bmatrix} e_0 \\ k \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ u \end{bmatrix}}_{\bar{u}}. \quad (30)$$

Cette construction est implémentée dans le solveur *solve_lwe_bdd*, car nous attaquons LWE directement via la résolution d'un problème de BDD. Nous pouvons le considérer ainsi car en trouvant l'erreur exacte $\mathbf{e}$ depuis un point $\mathbf{b}$ décalé, nous retrouvons le vecteur original $\mathbf{s}$. Comme les quantités B et $\bar{\mathbf{u}}$ sont connues, le solveur cherche à déterminer e en trouvant les bons coefficients pour le vecteur $[e_0^\top, k^\top]^\top$.

La matrice B est ensuite réduite à B' via LLL implémenté sous la librairie fypLLL. Compte tenu de cette réduction, les coefficients du vecteur $[e_0^\top, k^\top]^\top$ deviennent tous désorganisés : pour ce modèle, nous laissons la liberté à Gurobi de les déduire¹⁷. Nous nommons ces coefficients les $y_j \forall j \in \{1, 2, \dots, m\}$. Les m contraintes prennent alors la forme de $\mathbf{x} = B'\mathbf{y} + \bar{\mathbf{u}}$, et nous cherchons toujours à minimiser la norme euclidienne du vecteur d'inconnues $\mathbf{x}$. Soit l'ensemble $(\beta_{i,j}) \in \mathbb{Z}^{m \times m}$ l'ensemble des coefficients de la matrice B' et rappelons que $\bar{\mathbf{u}} = [\mathbf{0}_n^\top u^\top]$. Alors notre modèle devient le suivant (après avoir légèrement réorganisé les termes)

$$\begin{aligned} \text{minimiser : } & \sum_{i=1}^m x_i^2 \\ \text{sujet à : } & \sum_{j=1}^m \beta_{i,j} y_j + \bar{u}_i = x_i, & \forall i \in \{1, 2, \dots, m\} \\ & -t \leq x_i \leq t, & \forall i \in \{1, 2, \dots, m\} \\ & x_i, y_j \in \mathbb{Z} \end{aligned}$$

À titre de comparaison, prouvons l'équivalence entre le modèle de Shirase et le modèle BDD : transformons une instance du modèle de Shirase en une instance BDD. Rappelons que nous cherchons à extraire une matrice B telle que

$$B = \begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & q \cdot I_{(m-n) \times (m-n)} \end{bmatrix}$$

17. Nous pouvons considérer un potentiel réajustement de bornes comme un travail futur.

et que le modèle de Shirase prend la forme suivante :

$$\begin{aligned}
& \text{minimiser : } \sum_{k=1}^m x_k^2 \\
& \text{sujet à : } \sum_{j=1}^n w_{i,j} x_j - x_{n+i} + q f_i = -u_i, & \forall i \in \{1, 2, \dots, m-n\} \\
& -t \leq x_k \leq t, & \forall k \in \{1, 2, \dots, m\} \\
& - \left\lfloor \frac{t(nq - n + 1) + q - 1}{q} \right\rfloor \leq f_i \leq \left\lfloor \frac{t(nq - n + 1)}{q} \right\rfloor, & \forall i \in \{1, 2, \dots, m-n\} \\
& x_k, f_i \in \mathbb{Z}
\end{aligned}$$

Pour commencer, ajoutons pour les n composantes initiales du vecteur $\mathbf{x}$, soit les composantes de $\mathbf{x}_0$, les contraintes triviales suivantes

$$x_j = x_j \quad \forall j \in \{1, \dots, n\}.$$

Nous obtenons désormais alors bel et bien m contraintes tel que le fait le modèle BDD. En isolant x_{n+i} dans les contraintes des Shirase, nous trouvons :

$$x_{n+i} = \sum_{j=1}^n w_{i,j} x_j + q f_i + u_i, \quad \forall i \in \{1, 2, \dots, m-n\}$$

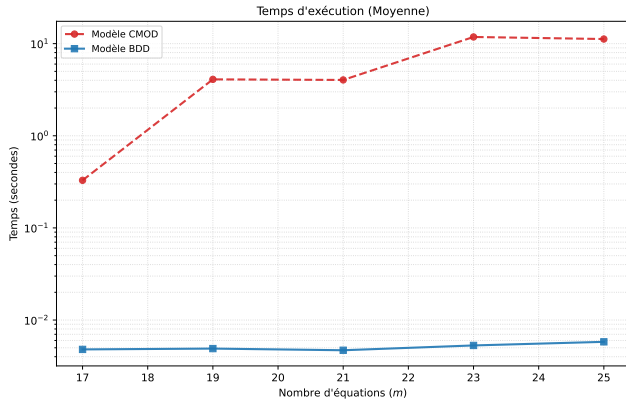
et en combinant ces équations avec les n contraintes triviales que nous venons de poser, il est possible de trouver le système suivant :

$$\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{x}_1 \end{bmatrix} = \begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & q \cdot I_{(m-n) \times (m-n)} \end{bmatrix} \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{f} \end{bmatrix} + \begin{bmatrix} \mathbf{0}_n \\ \mathbf{u} \end{bmatrix}.$$

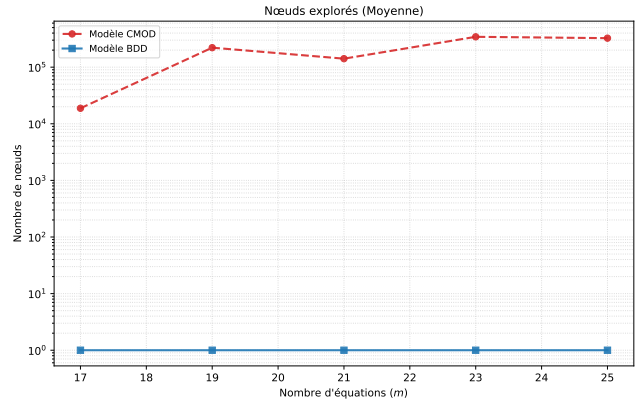
Le vecteur additionné à droite correspond exactement au vecteur $\bar{\mathbf{u}}$ de BDD, et la grande matrice est exactement la matrice encodant le réseau LWE que nous cherchions. En simplifiant cette dernière via LLL, nous retrouvons alors exactement le modèle BDD. Il est cependant nécessaire de remplacer le vecteur $[\mathbf{x}_0^\top, \mathbf{f}^\top]^\top$ par un vecteur y générique compte-tenu du désordre induit par l'application de LLL sur la matrice. Il serait intéressant de borner ce y dans un futur travail.

8.3.2 Étude de performance

Évaluons maintenant sa performance et ses limites en le comparant au solveur CMOD. Pour commencer, une série de tests à m variable avec $n = 11, q = 3329$ et $t = 2$ ont été effectués. Nous utilisons les mêmes 30 seeds qu'en figures 10 et 14. Les graphiques suivants présentent nos résultats.

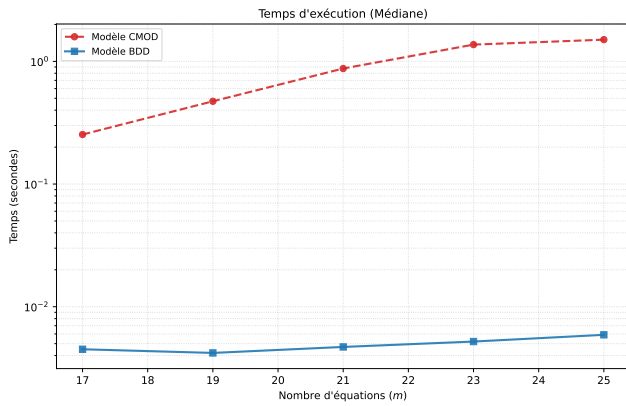


(a) Temps d'exécution

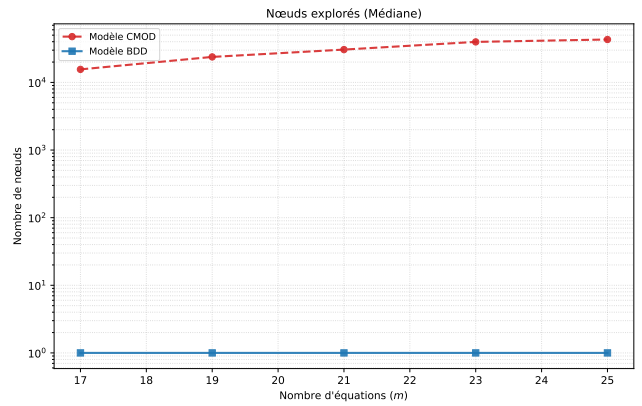


(b) Nombre de nœuds explorés

FIGURE 12 – Comparaison (moyenne) du modèle CMOD avec le modèle BDD.



(a) Temps d'exécution



(b) Nombre de nœuds explorés

FIGURE 13 – Comparaison (médiane) du modèle CMOD avec le modèle BDD.

Nous remarquons très rapidement que le modèle BDD est excessivement plus rapide notre ancien meilleur modèle, le modèle CMOD. Il n'y a même pas compétition : pour les quadruplets de paramètres que nous avons utilisé, le modèle BDD arrive à constamment résoudre les instances avec un seul noeud ! Nous pouvons donc en déduire que la résolution de ces problèmes est instantanée pour BDD. Le presolve de Gurobi combiné à la puissance de LLL nous permet de résoudre de petites instances excessivement rapidement. Qu'en est-il de nos anciennes instances limites cependant ? Rappelons qu'en figure 9, pour $n = 11$, $t = 4$, $q = 127$ et des m plus grand que 24, notre temps d'exécution tournait autour des 10^2 secondes avec de fréquents timeouts à 1300 secondes. Pour des problèmes identiques, notre nouveau solveur prends environ 10^{-2} secondes de moyenne, soit une amélioration d'environ 4 ordres de grandeur.

Remarque 20. Ces valeurs de temps d'exécution ne considère pas pour l'instant le temps de roulement de l'algorithme LLL. Cependant, ce dernier est négligeable compte tenu des dimensions des matrices sur lesquelles nous travaillons, et restera toujours polynomial. Sur de plus grosses instances, la difficulté du problème de PQNE dominera à coup sûr quant à l'évaluation de la complexité temporelle de notre pipeline de résolution.

Tentons maintenant de briser de plus grosses instances afin de déterminer les limites de notre nouveau solveur. Ramenons nous à $q = 3329$ et $t = 2$. Nous travaillerons cette fois-ci sur des augmentations progressives et simultanées de n et m . Par souci de temps et de reproductibilité, nous allons temporairement fixer un timeout à 250 secondes, et nous allons effectuer 3 tests (sur les seeds 1,2 et 3). Les valeurs moyennes et médianes présenteront les instances non résolues. Nous travaillons avec trois progressions différentes : $m = n + 20$, $m = n + 35$ et $m = n + 50$.

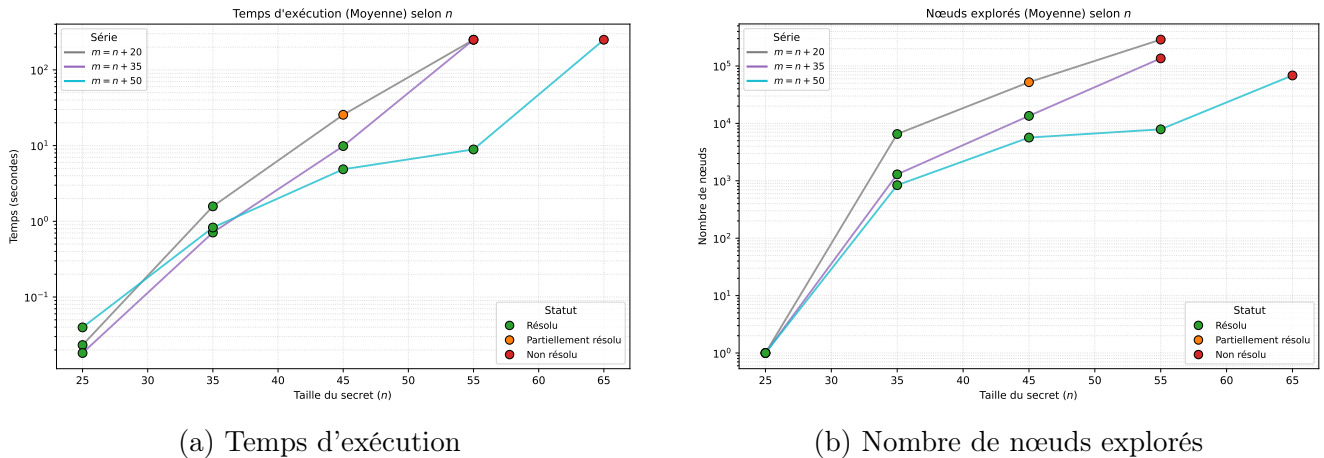


FIGURE 14 – Comparaison (moyenne) de trois séries de couples m, n différentes.

Remarquons que nous sommes capable de briser assez rapidement des instances beaucoup plus volumineuses que celles avec lesquelles nous travaillions plus tôt. Remarquons aussi qu'il est beaucoup plus facile pour Gurobi de résoudre des instances lorsqu'elles sont plus contraintes. Plus

la valeur de m est élevée par rapport à celle de n , pour il y a de chances que le modèle BDD puisse résoudre l'instance correspondant à ce couple m, n .

Notons qu'aucune des solutions a été trouvée en plus de 30 secondes : c'était soit moins, soit un timeout Gurobi. Nous supposons que cela vient d'une faiblesse dans la productivité de Gurobi : lorsque la base que lui donne LLL n'est pas assez orthogonale, il travaille excessivement fort pour retrouver la solution. Souvent, Gurobi semble se «perdre» dans l'espace des solutions. Le nombre d'itérations par noeud de branch-and-bound est approximativement deux fois plus élevé lorsque Gurobi ne trouve pas rapidement la solution. Remarquons aussi que plus la valeur de m augmente pour un certain n , plus le nombre de noeuds explorés diminue pour un même temps de recherche. En $n = 55$, les séries $+20$ et $+35$ ont toutes les deux timeout à 250 secondes, mais la série $n = m + 35$ fut capable d'itérer sur environ deux fois moins de noeuds que la série $n = m + 20$. L'augmentation de m par rapport à un n fixé est donc à double tranchant. Parfois, il est plus facile pour Gurobi de déterminer une solution via les contraintes supplémentaires, mais chaque noeud prend beaucoup plus de temps à être évalué.

Les tests effectués après $n = 65$ n'ont pas été représentés graphiquement puisqu'ils furent assez inconcluants, peu importe le terme additif x que nous ajoutions dans $n = m + x$. Le solveur BDD a réussi une unique fois à casser une instance de taille 125×75 , sur 50 essais. Cette solution fut trouvée en 9 secondes : le système aléatoire initial devait alors être assez bien conditionné pour LLL, qui retourna une base facile à utiliser pour Gurobi.

Remarque 21. Cette technique, bien que relativement efficace pour une ébauche, n'est malheureusement pas applicable aux instances de Kyber, compte tenu du fait que Kyber travaille avec des matrices A carrées, tandis que nous avons posé la condition nécessaire $m > n$. Il existe peut-être un moyen de transformer une instance carrée autrement afin de pouvoir la passer à un solveur MIP : cela pourrait être le sujet d'une prochaine étude.

9 Conclusion

9.1 Travail accompli

Cette étude consistait en une introduction à la cryptographie moderne pour étudiants au baccalauréat. En premier lieu, une revue de littérature de cryptographie classique a été rédigée. Cela nous a permis de mieux comprendre les faiblesses de cette dernière et les motivations derrière les récents développements en cryptographie post-quantique. Une attention particulière a été accordée à la cryptographie basée sur les réseaux. Le système LWE, acronyme de *Learning With Errors*, a été le point focal de l'étude.

L'article ayant inspiré ce travail en est un de Shirase [28]. Dans celui-ci, il est exhibé une réduc-

tion du problème Search-LWE, soit le problème LWE de base, à un problème de programmation en nombres entiers. À titre de nouveaux développements, le modèle de PQNE correspondant à LWE a été implémenté en python via le solveur d’optimisation Gurobi. Il a été possible de valider empiriquement la construction mathématique du modèle. Cependant, le solveur tel quel n’était pas très puissant : avec une erreur $t = 2$, un module $q = 127$ et une matrice de taille $m \times n$ étant de dimensions $\approx 35 \times 20$, le solveur se terminait en un timeout fixé à 25 minutes. Cependant, il est attendu que de telles petites instances de LWE soient solvable beaucoup plus rapidement.

L’amélioration principale ayant été apportée au modèle est celle d’une simplification de ce dernier utilisant l’algorithme LLL. L’algorithme LLL est un algorithme de réduction de base de réseau : il simplifie, raccourci et orthogonalise les vecteurs générant un réseau quelconque. En construisant notre réseau via une construction similaire à Shirase [28] trouvée en Jiang [8], il était possible d’incorporer une nouvelle base LLL-réduite dans un modèle de PQNE alternatif. Les améliorations apportées par ce changement furent sans équivoque : de telles petites instances, soit d’ordre 35×20 , furent maintenant résolubles en quelques millisecondes. Un mur fut cependant frappé à des instances telles que $n \approx 65$: le solveur tombait encore une fois dans un timeout fixé à 25 minutes. L’instance la plus grande résolue en un temps raisonnable fut une de taille 125×75 , avec $t = 2$ et $q = 3329$. Surprenamment, sa solution fut trouvée en moins de 10 secondes. Bien que cela soit une excellente amélioration, ces instances restent d’un ordre de difficulté excessivement bas lorsque comparées aux cryptosystèmes actuels basés sur les réseaux tels que Kyber et Dilithium.

9.2 Avenues futures

Il est possible de considérer cette contribution comme une base pouvant mener à d’avantage de recherche dans le domaine : rares sont les articles combinant la cryptographie post-quantique et l’optimisation. Ayant eu plus de temps sur l’été, nombreuses auraient été les améliorations ayant pu être apportées aux modèles de PQNE étudiés.

D’un point de vue d’optimisation pure, des techniques classiques viennent à l’esprit et auraient pu être utilisées sur chacun de nos deux modèles. D’abord, comme nous cherchons à minimiser une somme de carrés, il aurait été possible d’appliquer la linéarisation par morceaux, transformant la fonction objectif quadratique en un problème complètement linéaire. De plus, il aurait aussi pu être intéressant de déterminer manuellement des coupes pouvant affiner l’espace de recherche de Gurobi, possiblement à l’aide de d’inégalités valides fortes. Ces contraintes permettraient potentiellement de rapprocher l’espace de recherche à celui de l’espace de l’enveloppe convexe du problème. Notons aussi que notre modèle final proposé n’admet pas directement de bornes sur y , le travail étant laissé à Gurobi. Il serait évidemment intéressant de les déterminer manuellement.

D’autre part, le modèle initial, soit celui de Shirase, aurait pu être directement amélioré. Premièrement, suite à l’implémentation du modulo centré, il est possible de resserrer directement

les bornes sur f_i . Un nouveau calcul de ces dernières serait donc approprié. Sur le modèle final, troquer l'algorithme LLL pour l'algorithme BKZ, le standard d'industrie actuel (BKZ étant un LLL exécuté par blocs), aurait fort probablement pu améliorer la puissance de notre solveur. La base que retourne BKZ ayant une garantie d'orthogonalité et de vecteurs de petites tailles plus forte que LLL, il est naturel de supposer que l'espace engendré par une réduction BKZ aurait permis un travail beaucoup plus efficace de Gurobi. En dernier lieu, proposer une solution approximée initiale via l'algorithme de Babai roulant en temps polynomial à Gurobi pourrait fort probablement aider à retrancher une grande partie de l'arbre de recherche de celui-ci. Advenant le cas où Babai ne retourne pas la bonne solution, il serait possible d'utiliser la solution approchée comme borne supérieure, aidant grandement Gurobi à travailler plus vite.

Pour terminer, il serait intéressant d'analyser le fonctionnement précis de la résolution d'instance. Il a été possible de remarquer qu'avec le modèle optimisé, il n'a jamais fallu plus de 25 secondes à Gurobi pour déterminer une solution fonctionnelle. Le solveur fonctionnait donc un peu sous un principe de tout ou rien : soit la résolution échoue par manque de temps à 25 minutes, soit la solution est découverte excessivement rapidement. Il pourrait alors être intéressant de caractériser les instances à succès et d'observer leurs différences par rapport aux instances à échecs. Cela viendrait potentiellement expliquer le suprenant succès à 125×75 .

Références

- [1] Ajtai, M. (1996). Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108.
- [2] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., Stehlé, D., et al. (2019). Crystals-kyber algorithm specifications and supporting documentation. *NIST PQC Round*, 2(4) :1–43.
- [3] Babai, L. (1986). On lovász’lattice reduction and the nearest lattice point problem. *Combinatorica*, 6(1) :1–13.
- [4] Conforti, M., Cornuéjols, G., and Zambelli, G. (2014). *Getting Started*, pages 1–44. Springer International Publishing, Cham.
- [5] Diffie, W. and Hellman, M. E. (2022). New directions in cryptography. In *Democratizing cryptography : the work of Whitfield Diffie and Martin Hellman*, pages 365–390.
- [6] Espitau, T., Katsumata, S., Samwel, N., Wiggers, T., van Woerden, W., and Zijlstra, T. (2025). CRYPTREC : ML-KEM evaluation report. Evaluation Report CRYPTREC-EX-3502-2025, PQShield / Cryptography Research and Evaluation Committees (CRYPTREC).
- [7] Hallgren, S. and Vollmer, U. (2009). *Quantum computing*, pages 15–34. Springer Berlin Heidelberg, Berlin, Heidelberg.
- [8] Jiang, S. (2026). When the learning with errors problem meets the coherent ising machine : A penalty-free algorithm-hardware co-design. *arXiv preprint arXiv :2606.22843*.
- [9] Joy (2017). How long would it take to brute force an AES-128 key? Cryptography Stack Exchange. Accessed : 2026-07-23.
- [10] Kannan, R. (1987). Minkowski’s convex body theorem and integer programming. *Mathematics of operations research*, 12(3) :415–440.
- [11] Kitaev, A. Y. (1995). Quantum measurements and the abelian stabilizer problem. *Electron. Colloquium Comput. Complex.*, TR96.
- [12] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients.
- [13] Lenstra Jr, H. W. (1983). Integer programming with a fixed number of variables. *Mathematics of operations research*, 8(4) :538–548.
- [14] Lessnick, A. (2026). Implémentation de la réduction lwe et résolution via programmation en nombres entiers avec gurobi. https://github.com/alexless/solveur_lwe. Code source Python. Consulté le 10 juillet 2026.

- [15] Menezes, A. (2026). A gentle introduction to lattice-based cryptography. *Cryptology ePrint Archive*.
- [16] Menezes, A. J., Van Oorschot, P. C., and Vanstone, S. A. (1996). *Handbook of Applied Cryptography*. CRC press. Chapitre 4, Section 4.6.1 : Generating primary roots.
- [17] Micciancio, D. (2017). CSE 206A : Lattice Algorithms and Applications, Lecture 2 : Basis Reduction. Course lecture notes, University of California, San Diego – Department of Computer Science and Engineering.
- [18] National Institute of Standards and Technology (2024). Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication (FIPS) NIST FIPS 203, U.S. Department of Commerce.
- [19] Nguyen, P. (1999). Cryptanalysis of the goldreich-goldwasser-halevi cryptosystem from crypto’97. In *Annual International Cryptology Conference*, pages 288–304. Springer.
- [20] Peikert, C. (2009). Public-key cryptosystems from the worst-case shortest vector problem. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 333–342.
- [21] Peikert, C. (2013). Lattice-based cryptography : Short Integer Solution (SIS) and Learning With Errors (LWE). Lecture notes from the Summer School on Cryptography (crypt@b-it 2013). Georgia Institute of Technology.
- [22] Peikert, C. (2016). A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4) :283–424.
- [23] Rabin, M. O. (1980). Probabilistic algorithm for testing primality. *Journal of number theory*, 12(1) :128–138.
- [24] Regev, O. (2004). Quantum computation and lattice problems. *SIAM Journal on Computing*, 33(3) :738–760.
- [25] Regev, O. (2009). On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6) :1–40.
- [26] Regev, O. (2010). The learning with errors problem. *Invited survey in CCC*, 7(30) :11.
- [27] Schultz-Wu, M. (2021). Réponse à : Why do problems for post-quantum algorithms have to be np-hard? Cryptography Stack Exchange. (L’auteur est doctorant en cryptographie sur les réseaux à UCSD).
- [28] Shirase, M. (2023). Reduction of search-lwe problem to integer programming problem. *Cryptology ePrint Archive*.

- [29] Shor, P. W. (1999). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review*, 41(2) :303–332.
- [30] Silverman, J. H., Pipher, J., and Hoffstein, J. (2008). *An introduction to mathematical cryptography*, volume 1. Springer.
- [31] Smeets, I., Lenstra, A., Lenstra, H., Lovász, L., and van Emde Boas, P. (2010). *The History of the LLL-Algorithm*, pages 1–17. Springer Berlin Heidelberg, Berlin, Heidelberg.
- [32] Willsch, D., Hanussek, P., Hoever, G., Willsch, M., Jin, F., Raedt, H. D., and Michielsen, K. (2025). The state of factoring on quantum computers.
- [33] Wolsey, L. A. and Nemhauser, G. L. (2014). *Integer and combinatorial optimization*. John Wiley & Sons.

Annexes

A Implémentation de la réduction LWE

```
def solve_lwe(A, b, q, t, time_limit=1800):
    m, n = A.shape
    m_n = m - n
    A_0, A_1 = A[0:n, :], A[n:m, :]
    b_0, b_1 = b[:n], b[n:]
    A_0inv = np.array(Matrix(A_0).inv_mod(q)).astype(int)
    W = (A_1 @ A_0inv) % q
    u = (b_1 - W @ b_0) % q

    modele = gp.Model("SolveurLWE")
    modele.setParam('TimeLimit', time_limit)
    modele.setParam('SolutionLimit', 1)

    x = modele.addVars(m, vtype=GRB.INTEGER, lb=-t, ub=t, name="x")
    f_inf = -1 * ((t * (n * q - n + 1) + q - 1) // q)
    f_sup = (t * (n * q - n + 1)) // q
    f = modele.addVars(m_n, vtype=GRB.INTEGER, lb=f_inf, ub=f_sup, name="f")

    for i in range(m_n):
        cote_gauche = gp.quicksum(W[i, j] * x[j] for j in range(n)) - x[n+i] + q *
        f[i]
        modele.addConstr(cote_gauche == -u[i], name=f"eq_{i}")

    modele.setObjective(gp.quicksum(x[k] * x[k] for k in range(m)), GRB.MINIMIZE)
    modele.optimize()

    nodes, runtime = modele.NodeCount, modele.Runtime
    if modele.status in [GRB.OPTIMAL, GRB.TIME_LIMIT, GRB.SOLUTION_LIMIT] and
    modele.SolCount > 0:
        e_sol = np.array([int(x[k].X) for k in range(m)])
        s_hat = (A_0inv @ (b_0 - e_sol[:n])) % q
        return s_hat, e_sol, nodes, runtime

    return None, None, nodes, runtime
```